



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

DISEÑO E IMPLEMENTACIÓN DE UNA PLATAFORMA WEB PARA LA VENTA DE PRODUCTOS DIGITALES CON GESTIÓN DE LICENCIAS Y PAGOS INTEGRADOS

Joaquin Esperon Solari

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Año académico 2025-26

DISEÑO E IMPLEMENTACIÓN DE UNA PLATAFORMA WEB PARA LA VENTA DE PRODUCTOS DIGITALES CON GESTIÓN DE LICENCIAS Y PAGOS INTEGRADOS

Joaquin Esperon Solari

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

Año académico 2025-26

Paraules clau del treball: plataforma web, productos digitales, gestión de licencias, Next.js, Supabase, Stripe

Tutor: Cristina Suemay Manresa Yee

Gracias a mi familia, por el apoyo a lo largo de todos estos años.

A los profesores y profesoras de la EPS que han compartido su conocimiento y han estado presentes cuando se los necesitaba. Y a todo el personal de la UIB que trabaja cada día para que miles de estudiantes puedan estudiar en un entorno cuidado y agradable.

ÍNDICE

Índice	iii
Acrónimos	vii
Resumen	ix
1 Introducción	1
1.1 Objetivos	3
1.1.1 Objetivo principal	3
1.1.2 Objetivos específicos	3
1.1.3 Objetivos personales	4
1.2 Alcance	5
1.3 Estructura del documento	6
2 Metodología	7
2.1 Metodología de desarrollo	7
2.2 Planificación y fases del proyecto	8
2.3 Herramientas de desarrollo	10
2.4 Gestión de la calidad	11
3 Marco tecnológico	13
3.1 Plataformas de venta de productos digitales	13
3.2 Arquitecturas web para aplicaciones de comercio electrónico	15
3.2.1 Modelos de renderizado	15
3.2.2 Estilos de API	15
3.2.3 Arquitectura monolítica <i>serverless</i> vs. microservicios	15
3.3 Tecnologías del stack seleccionado	16
3.3.1 Next.js 15	16
3.3.2 TypeScript	16
3.3.3 Supabase	16
3.3.4 Stripe	17
3.3.5 Tailwind CSS y shadcn/ui	17
3.3.6 Resend	18
3.4 Seguridad en plataformas de comercio electrónico	18
3.5 Normativa aplicable	19
4 Análisis y diseño del sistema	21

4.1	Requisitos del sistema	21
4.1.1	Requisitos funcionales	21
4.1.2	Requisitos no funcionales	21
4.2	Arquitectura general del sistema	23
4.3	Modelo de datos	24
4.3.1	Planes como entidad propia	24
4.3.2	Desnormalización del campo <code>type</code>	24
4.3.3	Contador de activaciones y tabla de activaciones	25
4.3.4	Registro de eventos como log de auditoría	25
4.3.5	Precios en centavos enteros	25
4.3.6	Separación entre órdenes e ítems de orden	25
4.3.7	Tabla <code>profiles</code>	25
4.3.8	Identificadores públicos con <i>slug</i>	26
4.4	API pública de licencias	26
4.4.1	Semántica de verificar y activar	26
4.4.2	La clave de licencia como credencial	26
4.4.3	Identificador de máquina generado en el cliente	27
4.4.4	Limitación de tasa y política de caché	27
4.5	Diseño de la interfaz de usuario	27
4.5.1	Tienda pública	27
4.5.2	Flujo de compra	28
4.5.3	Panel de usuario	28
4.5.4	Visor de cursos	28
4.5.5	Panel de administración	28
4.6	Control de acceso y seguridad en el diseño	29
5	Implementación	31
5.1	Organización del código y convenciones	31
5.1.1	Clientes de Supabase según el contexto de ejecución	31
5.1.2	Tipos de base de datos escritos manualmente	31
5.1.3	Validación con Zod	32
5.2	Autenticación y gestión de sesiones	32
5.2.1	Flujo OAuth con PKCE y callback	32
5.2.2	Trigger de creación de perfil	33
5.2.3	Bug crítico: recursión infinita en las políticas RLS	33
5.3	Integración con Stripe	33
5.3.1	Bifurcación en el flujo de compra	33
5.3.2	Verificación HMAC de los webhooks	34
5.3.3	Diagnóstico de un cambio silencioso en la API de Stripe v2026	34
5.4	Ciclo de vida de licencias	34
5.4.1	Estados de una licencia	34
5.4.2	Generación de claves	35
5.4.3	Corrección del exploit de periodos de prueba	35
5.4.4	Estados de licencia ante fallos de pago	36
5.4.5	Sincronización de <code>cancel_at_period_end</code>	36
5.5	Entrega de contenido según el tipo de producto	36
5.5.1	Descarga de archivos con URL firmada	37

5.5.2	Validación de progreso en cursos	37
5.6	Comunicaciones y tareas programadas	37
5.6.1	Cinco eventos de correo transaccional	37
5.6.2	Arquitectura de dos pasos para el cron de trials	38
5.7	Incompatibilidad entre Edge Runtime y el cliente de Supabase	38
5.8	Patrón de dos consultas para licencias de administrador	39
5.9	Limitaciones técnicas identificadas	39
6	Pruebas y validación	41
6.1	Estrategia de pruebas	41
6.2	Verificación estática	42
6.3	Pruebas funcionales	42
6.4	Pruebas de seguridad	44
6.5	Pruebas de rendimiento	45
6.5.1	Prueba de carga sobre la Application Programming Interface (API) pública	45
6.5.2	Análisis con Lighthouse	45
6.6	Defectos detectados durante el desarrollo	46
6.7	Gestión de errores y casos límite	47
6.8	Limitaciones del proceso de pruebas	48
7	Resultados y discusión	51
7.1	Funcionalidades implementadas	51
7.2	Métricas de rendimiento	51
7.3	Capturas del sistema en funcionamiento	51
7.4	Comparación con soluciones existentes	51
7.5	Discusión	51
8	Conclusiones y trabajo futuro	53
8.1	Cumplimiento de objetivos	53
8.2	Conclusiones principales	53
8.3	Limitaciones del trabajo	53
8.4	Trabajo futuro	53
A	Manual de instalación y despliegue	55
A.1	Prerrequisitos	55
A.2	Configuración de variables de entorno	55
A.3	Instalación y ejecución local	55
A.4	Migraciones de base de datos	55
A.5	Despliegue en Vercel	55
B	Documentación de la API pública	57
B.1	Verificar licencia	57
B.2	Activar licencia	57
B.3	Desactivar licencia	57
B.4	Ejemplo de integración en Python	57
C	Esquema completo de la base de datos	59

ACRÓNIMOS

API Application Programming Interface

CRUD Create, Read, Update and Delete

CSR Client-Side Rendering

CSS Cascading Style Sheets

HMAC Hash-based Message Authentication Code

HTML HyperText Markup Language

HTTP Hypertext Transfer Protocol

IDOR Insecure Direct Object Reference

IVA Impuesto sobre el Valor Añadido

JSON JavaScript Object Notation

OAuth Open Authorization

OWASP Open Web Application Security Project

PKCE Proof Key for Code Exchange

REST Representational State Transfer

RLS Row Level Security

RSC React Server Components

SaaS Software as a Service

SEO Search Engine Optimization

SQL Structured Query Language

SSG Static Site Generation

SSR Server-Side Rendering

TFG Trabajo de Fin de Grado

TTL Time To Live

URL Uniform Resource Locator

UUID Universally Unique Identifier

RESUMEN

[Redactar el resumen en español aquí.]

Abstract

[Write the abstract in English here.]

INTRODUCCIÓN

Los productos digitales son bienes intangibles que se crean, distribuyen y consumen íntegramente en línea, sin inventario físico, sin logística de envío y sin necesidad de almacenamiento físico. Una vez producidos, pueden venderse un número ilimitado de veces sin coste adicional por unidad, lo que diferencia este modelo de cualquier negocio basado en bienes físicos. La categoría engloba productos muy distintos entre sí, como software y herramientas (aplicaciones, *plug-ins*, plantillas para herramientas como Notion o Google Sheets), libros electrónicos, cursos en línea, guías, contenido creativo (fotografías, ilustraciones, música, vídeos) o servicios de membresía como sitios web de acceso restringido, boletines o comunidades privadas, entre muchos otros formatos.

El mercado de productos digitales ha experimentado un crecimiento sostenido en los últimos años [1], impulsado por varios factores convergentes. La penetración del acceso a internet ha alcanzado cotas históricas a nivel global, lo que permite a un creador llegar a clientes en cualquier parte del mundo desde el primer día y sin infraestructura logística. Al mismo tiempo, la consolidación del trabajo remoto y la denominada *economía del creador* han llevado a un número creciente de profesionales a buscar modelos de negocio que puedan operarse íntegramente desde un ordenador, lo que casa de manera natural con la venta de productos digitales. A esto se suma una tendencia más sutil pero significativa: ante la sobreabundancia de información gratuita disponible en internet, lo que se vende con frecuencia ya no es la información en sí, sino su curación, organización y presentación. Buena parte de los cursos y guías digitales que se comercializan se sustentan en este principio.

Para el vendedor, los productos digitales presentan ventajas evidentes frente a los físicos. Carecen de coste de producción unitario: vender una copia o mil copias supone esencialmente el mismo gasto, dejando al margen la inversión en marketing. El stock es prácticamente infinito y la entrega al cliente es inmediata. La escalabilidad del negocio no exige ampliar la estructura productiva ni contratar personal adicional, sino únicamente alcanzar a más compradores. Estas características han propiciado la aparición de un ecosistema maduro de plataformas que ofrecen al creador la infraestructura

necesaria para vender sus productos sin desarrollarla por su cuenta.

Entre las plataformas más representativas se encuentran Gumroad [2], Lemon Squeezy [3], Payhip [4], Sellfy [5] y Whop [6]. Para tiendas más generales que también admiten productos digitales destaca Shopify. Todas ellas permiten al vendedor crear un perfil, publicar productos con sus descripciones e imágenes, configurar precios y términos de uso, y procesar los pagos a través de sus pasarelas integradas. Ofrecen además paneles con métricas básicas de ventas y herramientas elementales de marketing. Su valor principal está en abstraer al vendedor de la complejidad técnica que implica montar una infraestructura de comercio electrónico, especialmente la gestión de pagos y el cumplimiento normativo.

Este modelo, sin embargo, presenta limitaciones que se hacen más visibles a medida que el negocio crece o cuando se utiliza para casos de uso menos convencionales. La primera limitación es de orden económico: las comisiones son significativas. Gumroad aplica un 10 % más una tarifa fija de 0,50 \$ por transacción para ventas a través del perfil propio del vendedor, porcentaje que asciende al 30 % cuando la venta llega a través del descubrimiento dentro de su *marketplace* [7]. Lemon Squeezy aplica un 5 % más 0,50 \$ por transacción [8]. Para volúmenes de venta elevados, este coste recurrente justifica por sí solo la inversión inicial en una solución propia.

La segunda limitación afecta al control sobre los datos del negocio. Las plataformas centralizadas exponen al vendedor únicamente la información que ellas deciden, a través de sus APIs, con los filtros y formatos que ellas definen. No es posible ejecutar consultas arbitrarias sobre los propios datos, cruzarlos con fuentes externas dentro de la plataforma o acceder a metadatos internos. Más allá de la limitación técnica, existe un riesgo operativo concreto: si la plataforma cambia sus términos, suspende una cuenta o cesa su actividad, el vendedor pierde el acceso a su histórico. En la práctica se han dado casos en ambas direcciones: modificaciones de modelo de precios sin previo aviso [9] y suspensiones temporales de cuentas que dejaron a los afectados sin acceso a sus datos durante el período de revisión [10].¹

La tercera limitación, y la que motivó directamente el desarrollo del presente trabajo, es la falta de soporte adecuado para productos de software con licencias activables por dispositivo. Cuando un vendedor distribuye un programa que el cliente debe ejecutar en su propio equipo (un *script* en Python, un *bot* de *trading* escrito en MQL5 (*MetaQuotes Language 5*), una aplicación de escritorio), surge la necesidad de un sistema que permita verificar desde el propio software si el usuario tiene una licencia válida y limitar el número de máquinas en las que puede activarse. Algunas plataformas exponen mecanismos de verificación de licencias, pero ninguna cubre el ciclo de vida completo que requiere la distribución de software. Whop, una plataforma con creciente cuota de mercado, ofrece una API de verificación servidor a servidor que permite consultar el estado de un pase de membresía desde un proceso externo como un *expert advisor* en MetaTrader 5 (plataforma de análisis y *trading* algorítmico de MetaQuotes Software); sin embargo, su modelo está orientado a la gestión de membresías y accesos a comunidades, no al control de activaciones por dispositivo. La plataforma no contempla el registro del número de máquinas en las que se ejecuta el software, la desactivación

¹ Entre los casos reportados por la comunidad de vendedores digitales se encuentran la eliminación del plan gratuito de Gumroad en mayo de 2023 y diversas suspensiones de cuentas en Lemon Squeezy durante 2024, documentadas en foros y redes sociales por los propios afectados.

remota de una instalación concreta ni un histórico auditable de eventos de activación por licencia. Gumroad ofrece igualmente un *endpoint* público de verificación de claves, pero adolece de las mismas carencias: no permite limitar el número de activaciones por máquina, bloquear una clave desde un panel de administración ni consultar un registro de eventos asociado a cada licencia.

A todo lo anterior se suma una motivación de carácter más personal. A título personal, desarrollo y comercializo programas de *trading* algorítmico escritos en MQL5, que se ejecutan como *expert advisors* dentro de MetaTrader 5. La intención de comercializar este tipo de producto fue el detonante inicial del proyecto: ante la necesidad de un sistema de verificación de licencias, me planteé si tenía sentido pagar comisiones recurrentes a una plataforma intermediaria o desarrollar una solución propia que cubriera tanto las necesidades inmediatas como las futuras. Adicionalmente, identifiqué un segundo caso de uso dentro del entorno cercano: un proyecto familiar de venta de cursos, manuales y planos para la construcción de estructuras de madera en jardín (huertos elevados, casetas para almacenaje, fogoneros, mini-casas), susceptible de utilizar la misma plataforma para vender productos de tipo curso, libro electrónico y plantilla. Esta diversidad de casos de uso reales reforzó la decisión de plantear una plataforma con soporte heterogéneo para varios tipos de producto digital, en lugar de una solución específica para un único formato.

1.1 Objetivos

1.1.1 Objetivo principal

El objetivo principal del presente Trabajo de Fin de Grado (TFG) es el diseño y la implementación de una plataforma web para la venta de productos digitales que integre, sin depender de servicios de terceros para los aspectos críticos del negocio, la gestión del ciclo de vida completo de las licencias, una pasarela de pagos consolidada, la entrega segura de archivos y un panel de administración propio. La plataforma debe dar soporte específico a cuatro tipos de producto con requisitos distintos: software con licencias activables por dispositivo, libros electrónicos, plantillas y cursos en línea. El resultado entregado al final del trabajo es una plataforma funcional, desplegada en un entorno de producción, que pueda servir como base reutilizable para distintos negocios de venta de productos digitales mediante personalización de la marca y los contenidos.

1.1.2 Objetivos específicos

A partir del objetivo principal se han definido los siguientes objetivos específicos, formulados de manera que su consecución pueda verificarse de forma objetiva al finalizar el trabajo:

1. **OE-01.** Implementar un sistema de autenticación de usuarios con soporte para credenciales de correo electrónico y contraseña, autenticación mediante Open Authorization (OAuth) con Google y un flujo completo de recuperación de contraseña.

2. **OE-02.** Diseñar un catálogo público de productos con soporte para cuatro tipos diferenciados (*software*, *ebook*, *template* y *course*), cada uno con su propio comportamiento posterior a la compra.
3. **OE-03.** Integrar una pasarela de pagos del mercado con soporte para pagos únicos, suscripciones recurrentes con periodicidad mensual o anual y períodos de prueba con o sin recogida previa de tarjeta.
4. **OE-04.** Implementar un sistema de gestión de licencias con ciclo de vida completo (emisión, activación por dispositivo, suspensión por impago, recuperación, cancelación, expiración y revocación) y un histórico auditable de todos los eventos asociados a cada licencia.
5. **OE-05.** Desarrollar una API Representational State Transfer (REST) pública que permita a aplicaciones de terceros, sin sesión de usuario, verificar la validez de una licencia, registrar la activación en una nueva máquina y desactivar una máquina previamente registrada, con limitación de tasa para prevenir abusos.
6. **OE-06.** Garantizar la entrega segura de los archivos asociados a cada producto mediante Uniform Resource Locators (URLs) firmadas con tiempo de expiración limitado, accesibles únicamente para usuarios con una licencia activa para dicho producto.
7. **OE-07.** Implementar un visor integrado de cursos en línea con soporte para vídeo embebido (YouTube y Vimeo), contenido textual con formato Markdown, archivos adjuntos por lección y seguimiento del progreso del usuario.
8. **OE-08.** Diseñar un sistema de notificaciones por correo electrónico transaccional para los eventos clave del ciclo de vida de las licencias: confirmación de compra, fallo de pago, recuperación de pago, cancelación de suscripción y expiración inminente del periodo de prueba.
9. **OE-09.** Desarrollar un panel de administración que permita la gestión completa de productos, planes de precio, currículos de cursos, licencias, órdenes, cupones de descuento, reseñas y clientes.

1.1.3 Objetivos personales

Más allá del resultado técnico, este trabajo persigue un conjunto de objetivos personales relacionados con la consolidación de las competencias adquiridas a lo largo del grado y con la incorporación de tecnologías de uso actual en el entorno profesional.

En primer lugar, se busca aplicar los conocimientos adquiridos en *Bases de Datos I* (21716) y *Bases de Datos II* (21726) sobre un esquema relacional no trivial, en el que decisiones como el diseño de claves primarias compuestas, la elección de índices de rendimiento, las restricciones de integridad referencial o el control de acceso a nivel de fila tienen consecuencias directas y medibles sobre el comportamiento del sistema. En segundo lugar, se persigue poner en práctica lo aprendido en *Aplicaciones Distribuidas en Internet e Interfaces de Usuario* (21724) sobre un *framework* de desarrollo web actual, en el que el frontend y el backend coexisten en un mismo proyecto, con renderizado del

lado del servidor, comunicación con servicios externos y consideraciones de seguridad propias del desarrollo web profesional. En tercer lugar, la asignatura *Gestión de Proyectos* (21723) ofreció el marco para abordar la planificación del desarrollo de un sistema de envergadura considerable, con múltiples fases interdependientes y la necesidad de adaptar el plan inicial a problemas técnicos no previstos.

A nivel tecnológico, se buscó deliberadamente trabajar con un *stack* representativo del estado actual del desarrollo web profesional. Esto incluye Next.js como *framework* de aplicación con renderizado del lado del servidor y API *routes* integradas, TypeScript como lenguaje de programación para todo el proyecto, Supabase como plataforma *backend-as-a-service* basada en PostgreSQL, Stripe como pasarela de pagos y Tailwind CSS junto con *shadcn/ui* para la construcción de la interfaz de usuario. Profundizar en estas tecnologías, y especialmente en la integración con una pasarela de pagos del mercado, formaba parte de los intereses formativos del autor desde el inicio del proyecto, dada la relevancia de los pagos en línea como industria y su presencia transversal en gran parte del software comercial.

1.2 Alcance

El alcance del presente trabajo se centra en el diseño, la implementación y el despliegue de una plataforma web completa para la venta de productos digitales por parte de un único vendedor o administrador. El sistema cubre todo el ciclo desde la publicación de un producto hasta su consumo por parte del cliente final: gestión del catálogo, pasarela de pagos, generación y administración de licencias, entrega segura de archivos, visor de cursos integrado, notificaciones por correo electrónico y panel de administración. Los pagos se procesan a través del modo de pruebas de Stripe, suficiente para validar el flujo íntegro de compra, suscripciones, períodos de prueba y *webhooks* sin manejar dinero real. La plataforma se despliega sobre Vercel para el código de aplicación y Supabase para la base de datos, autenticación, almacenamiento de archivos y funciones programadas, con envío real de correos electrónicos a través de Resend.

Quedan fuera del alcance del trabajo varios aspectos que se identifican explícitamente para delimitar el ámbito de la evaluación. La plataforma está concebida para un único vendedor, por lo que no se implementa un modelo de *marketplace* con múltiples vendedores independientes. No se desarrolla aplicación móvil nativa: el acceso es exclusivamente web, si bien la interfaz se adapta a pantallas de tamaño reducido. No se aborda el cálculo y la recaudación automática de impuestos por jurisdicción geográfica (Impuesto sobre el Valor Añadido (IVA), *sales tax*), funcionalidad que Stripe ofrece como producto adicional pero cuya integración trasciende el alcance académico de este trabajo. No se han implementado pruebas unitarias ni de integración automatizadas: la validación se ha realizado de forma manual y *end-to-end*, dado que la mayor parte de la lógica del sistema reside en la interacción con servicios externos (Supabase, Stripe) cuyo comportamiento no resulta sencillo de aislar mediante *mocks* sin perder fidelidad respecto al sistema real. La interfaz de usuario está disponible únicamente en inglés; no se implementa internacionalización. Por último, el panel de administración no incluye funcionalidades de *analytics* avanzado con gráficos de evolución temporal de ingresos, un sistema de afiliados o referidos, ni soporte de chat o tickets de incidencias integrado.

1.3 Estructura del documento

La presente memoria se organiza en ocho capítulos. Tras el presente capítulo introductorio, el capítulo 2 describe la metodología de desarrollo adoptada, las fases en las que se ha estructurado el trabajo y las herramientas utilizadas durante el proceso. El capítulo 3 presenta el marco tecnológico, analiza con mayor profundidad las plataformas existentes en el mercado y justifica las decisiones técnicas adoptadas en cuanto al *stack* de desarrollo. El capítulo 4 aborda el análisis de requisitos y el diseño del sistema, incluyendo la arquitectura general, el modelo de datos, el diseño de la API y el diseño de la interfaz de usuario. El capítulo 5 describe la implementación de las funcionalidades clave del sistema, deteniéndose en las decisiones técnicas más relevantes y en los problemas resueltos durante el desarrollo. El capítulo 6 expone la estrategia de pruebas seguida y las métricas de rendimiento obtenidas. El capítulo 7 presenta los resultados del trabajo, incluyendo la comparación entre las funcionalidades planificadas y las finalmente entregadas. Por último, el capítulo 8 recoge las conclusiones del trabajo, identifica las limitaciones detectadas y propone líneas de trabajo futuro.

CAPÍTULO 2

METODOLOGÍA

Este capítulo describe el proceso de trabajo seguido durante el desarrollo del proyecto: el enfoque metodológico adoptado, la planificación efectiva en fases, las herramientas utilizadas como soporte al desarrollo y los mecanismos aplicados para garantizar la calidad del sistema resultante. Las tecnologías concretas que componen la plataforma se presentan en el capítulo 3; aquí se aborda el *cómo* se trabajó, no el *con qué* se construyó.

2.1 Metodología de desarrollo

El desarrollo siguió un enfoque iterativo e incremental, sin adopción de una metodología formal específica como Scrum, Kanban o XP. La decisión responde a las características concretas del proyecto: un único desarrollador, alcance acotado en el tiempo y ausencia de un equipo de trabajo. Bajo estas condiciones, el coste asociado a las ceremonias y la gestión propias de un marco formal (planificación de *sprints*, reuniones periódicas, documentación exhaustiva de tareas, métricas de equipo) no se justifica frente al beneficio que aportan en proyectos de mayor envergadura.

En su lugar, se adoptó un ciclo de trabajo organizado en torno a la entrega de funcionalidades completas. Cada iteración consistía en identificar una funcionalidad necesaria, analizar las opciones de diseño, implementarla hasta dejarla operativa y, antes de pasar a la siguiente, validarla manualmente sobre la aplicación en ejecución. Las dependencias técnicas entre funcionalidades dictaron el orden natural del desarrollo: en primer lugar la infraestructura básica, sobre ella la autenticación, después el catálogo público, a continuación los pagos y la gestión de licencias, y así sucesivamente hasta llegar a las funcionalidades más avanzadas. Este orden no se fijó formalmente desde el inicio, sino que se reveló como el único razonable a medida que cada bloque construía sobre los anteriores.

Para el seguimiento del progreso se optó por un sistema mínimo basado en un fichero `todo.md` mantenido dentro del propio repositorio. Cada funcionalidad pendiente se

registraba como una entrada de la lista; al completarse se marcaba como terminada, y las nuevas ideas que iban surgiendo durante el desarrollo se añadían al final. Esta opción presenta ventajas evidentes respecto a herramientas externas de gestión: no requiere ninguna aplicación adicional, queda versionada junto al código, resulta inmediatamente accesible desde el editor y permite editar el estado del proyecto con la misma fluidez con la que se edita cualquier fichero de texto. Como contrapartida, no conserva información temporal precisa (fechas de creación o finalización de tareas) ni permite consultar el cronograma exacto del desarrollo. Para un proyecto unipersonal de duración acotada, la simplicidad resultaba más valiosa que el detalle de la trazabilidad.

2.2 Planificación y fases del proyecto

La planificación del proyecto no se articuló como un documento de requisitos cerrado previo al desarrollo, sino como un proceso de refinamiento continuo del alcance. La idea de partida era acotada: construir una plataforma web para la venta de productos digitales con soporte propio para verificación de licencias, sobre el *stack* tecnológico que se justifica en el capítulo 3. El alcance concreto (comportamiento exacto del panel de administración, estructura del *dashboard* del usuario final, flujos específicos asociados a cada tipo de producto) fue tomando forma a medida que avanzaba el desarrollo y se identificaban necesidades reales al uso del sistema.

A posteriori, el desarrollo puede estructurarse en una fase inicial de nueve etapas consecutivas que construyeron la plataforma base, seguida de un período de refinamientos iterativos en el que se incorporaron funcionalidades adicionales y se corrigieron limitaciones detectadas durante el uso del sistema. Las nueve etapas iniciales, dictadas por las dependencias técnicas entre los distintos componentes, fueron las siguientes:

1. **Infraestructura base.** Configuración del *stack* tecnológico, definición del esquema inicial de la base de datos, sistema de tipos sobre las tablas de Supabase y separación entre los clientes de servidor y de navegador.
2. **Autenticación y *layout*.** Sistema de registro y acceso, integración del flujo OAuth con Google, *middleware* de protección de rutas y estructura básica de la aplicación.
3. **Administración del catálogo.** Create, Read, Update and Delete (CRUD) de productos y planes de precio desde el panel de administración, con soporte para los distintos tipos de producto.
4. **Tienda pública.** *Storefront* público, página de listado de productos y página de detalle con presentación de los planes asociados.
5. **Pagos y generación de licencias.** Integración con la pasarela de pagos, implementación de los manejadores de eventos de pago, generación de claves de licencia y soporte para pagos únicos y períodos de prueba iniciales.
6. **API pública de licencias.** *Endpoints* de verificación, activación y desactivación con limitación de tasa, pensados para ser consumidos desde software de terceros sin sesión de usuario.

7. **Dashboard de usuario.** Vista de licencias y órdenes, perfil de usuario y descarga de archivos asociados a las compras.
8. **Panel de administración completo.** Gestión avanzada de licencias (revocación, suspensión, recuperación), órdenes, cupones de descuento, reseñas y clientes.
9. **Escalabilidad y rendimiento.** Definición de índices sobre las tablas con mayor volumen de consulta, política de cacheado en la API pública y pruebas de carga para validar los objetivos de latencia.

Finalizadas estas etapas, el desarrollo continuó durante varias semanas en un modo de iteración incremental, en el que se incorporaron funcionalidades que no estaban previstas en la planificación inicial y se reelaboraron componentes que no se ajustaban completamente al uso esperado. Entre las funcionalidades añadidas en esta fase destacan el visor integrado de cursos en línea, la cancelación de suscripciones por parte del usuario, el manejo de pagos fallidos y la suspensión automática de licencias afectadas, el sistema de notificaciones transaccionales por correo electrónico y la función programada de avisos de expiración del período de prueba.

El carácter emergente de los requisitos se manifestó de diversas formas a lo largo del proceso. Algunas funcionalidades no estaban previstas al inicio y se incorporaron al detectarse su utilidad: el visor integrado de cursos, por ejemplo, fue una decisión tomada durante el proyecto al comprobar que su implementación sobre vídeos embebidos de YouTube y Vimeo era razonablemente sencilla y aportaba valor diferencial frente a las plataformas analizadas. Otras se replantearon a mitad del proceso. El caso más representativo es la funcionalidad que permitía a los usuarios descartar sus propias licencias gratuitas y de prueba: se implementó por completo y permaneció varios días en el sistema, pero un análisis posterior reveló dos problemas. Por un lado, introducía complejidad significativa en el control de los períodos de prueba, al permitir a un usuario reclamar repetidamente el mismo *trial* descartando el anterior. Por otro, no se corresponde con el comportamiento estándar de las plataformas analizadas, en las que una licencia gratuita simplemente permanece en la biblioteca del usuario hasta que este deja de usarla. Estas consideraciones llevaron a eliminar la funcionalidad y a simplificar el modelo de gestión de licencias.

Hubo igualmente componentes que necesitaron reescritura tras una primera implementación. El *middleware* de protección de rutas administrativas, por ejemplo, se implementó inicialmente con el cliente de base de datos estándar, pero fallaba sin producir ningún error visible (la consulta al perfil del usuario devolvía vacío sin lanzar ninguna excepción) al combinarse con sesiones procedentes del flujo OAuth; resolverlo obligó a reescribir la verificación mediante llamadas directas a la API de la base de datos usando una clave de acceso privilegiado. El detalle técnico de esta solución se presenta en el capítulo 5. De forma similar, el sistema de períodos de prueba pasó por varias iteraciones hasta diferenciar correctamente los *trials* gestionados internamente de los *trials* nativos de Stripe con recogida diferida de tarjeta. Estos episodios, característicos de un proceso iterativo en el que las decisiones de diseño se validan contra el sistema en funcionamiento, se desarrollan con mayor detalle en el capítulo 5.

En cuanto a la dimensión temporal, el proyecto se inició en enero de 2026 con una fase de análisis, diseño e investigación tecnológica. La implementación se desarrolló de manera concentrada durante los meses de marzo y abril de 2026, aproximadamente

2. METODOLOGÍA

ocho semanas de trabajo desde la configuración inicial del *stack* hasta la finalización de una versión completa de la plataforma. A partir de mayo, el esfuerzo se centró en la redacción de la presente memoria y en la incorporación de mejoras puntuales detectadas durante este proceso, sin abordar nuevas funcionalidades de gran calado. La convocatoria de entrega del 3 de julio de 2026 actuó como restricción temporal global, obligando a fijar un punto de corte en la implementación con anterioridad suficiente para dedicar el tiempo necesario a la elaboración de la memoria. Estimando el esfuerzo total, el trabajo suma aproximadamente 450 horas: unas 60 horas de análisis y diseño previo (enero–febrero), 200 horas de implementación (marzo–abril), 50 horas de pruebas y depuración a lo largo del desarrollo, y 140 horas dedicadas a la redacción y revisión de la presente memoria (mayo–julio). La distribución aproximada de las etapas sobre la totalidad del periodo del trabajo se recoge en la figura 2.1.

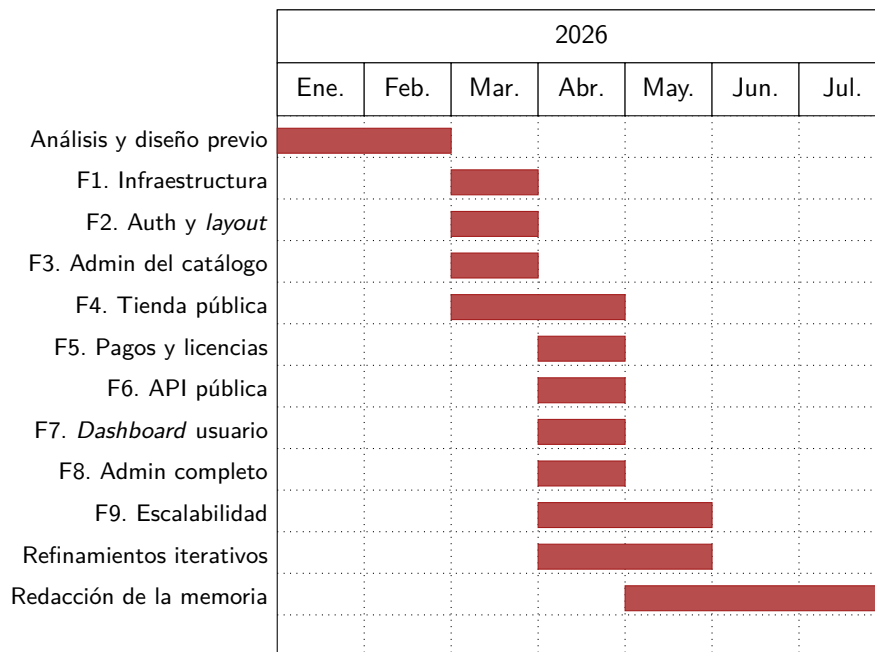


Figura 2.1: Distribución aproximada de las fases del trabajo a lo largo del período de desarrollo. Las duraciones son orientativas; el solapamiento entre etapas de implementación refleja su ejecución consecutiva más que su duración exacta en semanas.

Durante el desarrollo se mantuvo un *backlog* de funcionalidades que se decidió dejar fuera del alcance de la primera versión. La comparación entre las funcionalidades planificadas y las efectivamente entregadas se presenta en el capítulo 7.

2.3 Herramientas de desarrollo

El entorno de trabajo se construyó alrededor de un conjunto reducido de herramientas estándar en el desarrollo web actualmente. Las tecnologías que componen el *stack* de la aplicación (Next.js, Supabase, Stripe y las demás) se analizan y justifican en el capítulo 3; esta sección se limita a las herramientas de soporte al proceso de desarrollo. Como editor se utilizó Visual Studio Code [11], configurado con las extensiones habituales

para TypeScript, Tailwind CSS y resaltado de sintaxis Structured Query Language (SQL). El control de versiones se gestionó con Git [12], con el repositorio alojado en GitHub. Dado el carácter unipersonal del proyecto, no se siguió un modelo de ramas paralelas: el trabajo se realizó directamente sobre la rama principal, con *commits* frecuentes que constituyen el histórico completo del desarrollo. La motivación principal para mantener el repositorio en remoto no fue, por tanto, la colaboración, sino la posibilidad de continuar el trabajo desde distintas máquinas y disponer de una copia de seguridad accesible.

Para el desarrollo local resultó especialmente útil la integración con servicios externos a través de herramientas dedicadas. La *CLI* oficial de Stripe (`stripe listen`) permite reenviar al servidor en ejecución local los eventos del modo de pruebas mediante un túnel autenticado, lo que resulta indispensable para depurar la lógica de los *webhooks* sin necesidad de desplegar la aplicación en cada iteración. Para la base de datos, el almacenamiento de archivos y las funciones programadas se utilizó el panel web de Supabase, que ofrece tanto un editor SQL como una interfaz para inspeccionar tablas y registros en tiempo real.

Durante el desarrollo se hizo uso de un asistente de inteligencia artificial generativa (Claude, de Anthropic) como apoyo en tareas concretas: generación de fragmentos de código, depuración de errores técnicos, consulta sobre las APIs de los servicios integrados (Supabase, Stripe, Resend) y resolución de dudas puntuales sobre el uso del *framework*. Las decisiones de diseño, la arquitectura del sistema y la selección de las funcionalidades a desarrollar correspondieron al autor del trabajo.

Como herramienta de análisis estático del código se mantuvo la configuración por defecto que proporciona Next.js, basada en ESLint [13] con los conjuntos de reglas *core-web-vitals* (centradas en buenas prácticas de rendimiento web) y *typescript* (específicas del lenguaje). No se introdujeron reglas personalizadas adicionales: la configuración estándar resultó suficiente para detectar los errores que aparecen con mayor frecuencia durante el desarrollo.

2.4 Gestión de la calidad

Sin un proceso formal de pruebas automatizadas, la calidad del sistema descansó en varios mecanismos complementarios aplicados durante el desarrollo.

El mecanismo con mayor impacto fue el uso de TypeScript en todo el código del proyecto, junto con la definición manual de los tipos correspondientes a las tablas de Supabase. Esta práctica permite detectar en tiempo de compilación una clase importante de errores que de otro modo solo se manifestarían en ejecución: nombres de columnas incorrectos, valores nulos no contemplados, formatos de respuesta inconsistentes entre el cliente y el servidor. De manera complementaria, la validación de los datos de entrada se aborda mediante esquemas Zod¹ en los principales formularios del cliente y en las rutas de API que reciben datos del usuario, garantizando que los datos procesados respondan al formato esperado antes de llegar a la lógica de negocio.

¹Zod [14] es una librería de validación de esquemas para TypeScript que permite declarar la estructura y los tipos esperados de un valor y verificar su conformidad en tiempo de ejecución, antes de que el dato llegue a la lógica de negocio.

Las pruebas funcionales se realizaron de forma manual y *end-to-end* sobre el sistema en ejecución local. Al finalizar la implementación de cada funcionalidad, se ejecutaba la aplicación con `npm run dev` y se validaba el flujo completo desde la interfaz, prestando especial atención a la consistencia entre el rol de administrador y el rol de usuario final. Resulta habitual, por ejemplo, verificar que un cambio aplicado desde el panel de administración se refleja correctamente en el *dashboard* del usuario, o que un nuevo evento generado por un *webhook* de Stripe se traduce en el comportamiento esperado en la base de datos y en la interfaz. Esta validación se complementaba con pruebas dirigidas a casos extremos: valores límite, datos inesperados en formularios o intentos de acceso a recursos pertenecientes a otro usuario. No se implementaron pruebas automatizadas, unitarias o de integración: una limitación que se reconoce explícitamente y que se discute en el capítulo 8.

El desarrollo se llevó a cabo sin un entorno de *staging* intermedio. Toda la implementación se ejecutaba localmente, con Stripe operando en su modo de pruebas y con un proyecto de Supabase dedicado al desarrollo. Esta decisión, propia de un proyecto unipersonal de duración acotada, resulta razonable mientras el sistema se valide directamente contra los servicios externos en sus modos de prueba, que reproducen con fidelidad el comportamiento de producción. El despliegue en Vercel y la transición de Stripe al modo real quedan reservados para la entrega final del trabajo.

En cuanto a la seguridad, la estrategia consistió en delegar los componentes más críticos a servicios especializados y centrar el esfuerzo propio en los controles que corresponden al código de aplicación. El almacenamiento de contraseñas y el procesamiento de datos de tarjeta quedan en manos de Supabase y Stripe respectivamente, servicios que cumplen los estándares del sector. El código propio garantiza que cada usuario accede únicamente a sus propios recursos, verifica la autenticidad de los eventos de pago entrantes, controla la entrega de archivos digitales y usa la base de datos como segunda línea de defensa ante posibles errores en el código. Las medidas concretas que implementan estos principios se presentan en el capítulo 3.

MARCO TECNOLÓGICO

3.1 Plataformas de venta de productos digitales

El mercado de plataformas para la distribución de productos digitales es amplio y relativamente maduro. Estas soluciones ofrecen al creador una infraestructura lista para usar que incluye la gestión del catálogo, el procesamiento de pagos y la entrega de los archivos, con el objetivo de abstraer la complejidad técnica de operar una tienda en línea. Su atractivo principal es la rapidez de configuración: en menos de una hora pueden estar operativos una cuenta, los productos publicados y la pasarela de pagos configurada.

Gumroad [2] es una de las plataformas más veteranas del sector, diseñada para creadores individuales. Cobra una comisión del 10 % más 0,50 \$ por transacción cuando la venta llega a través del perfil propio del vendedor, porcentaje que asciende al 30 % en ventas procedentes de su *marketplace* [7]. Dispone de un *endpoint* público de verificación de claves de licencia (GET /v2/licenses/verify), pero su modelo no contempla el registro de activaciones por dispositivo: un mismo código puede emplearse en un número arbitrario de máquinas sin que la plataforma lo detecte ni lo limite.

Lemon Squeezy [3] nació orientado a la distribución de Software as a Service (SaaS), con soporte nativo para suscripciones recurrentes y gestión automática de impuestos. Aplica una comisión del 5 % más 0,50 \$ por transacción [8]. Dispone de una API de licencias más elaborada que Gumroad, aunque tampoco incluye el concepto de activaciones por dispositivo como unidad gestionable desde el panel de administración.

Payhip [4] está orientado principalmente a libros electrónicos, cursos y descargas simples. Su plan gratuito aplica una comisión del 5 % por transacción; los planes de pago eliminan las comisiones a cambio de una cuota mensual fija. No expone una API pública de verificación de licencias.

Sellfy es una plataforma de suscripción mensual (desde 29 \$/mes) orientada a la venta de descargas digitales, vídeos y suscripciones de contenido. A diferencia de Gumroad o Payhip, no aplica comisión por transacción en sus planes de pago, pero

3. MARCO TECNOLÓGICO

exige la cuota mensual independientemente del volumen de ventas. No ofrece API pública de verificación de licencias.

Whop [6] es una plataforma con creciente presencia en el mercado, concebida para la gestión de membresías y comunidades. Ofrece una API de verificación servidor a servidor que permite consultar el estado de un pase de membresía mediante una clave y la clave de API del vendedor, sin requerir sesión activa del usuario final. Esta característica hace técnicamente posible integrar la verificación desde un proceso externo. Sin embargo, su modelo está orientado a membresías, no al control de activaciones por dispositivo: la plataforma no almacena en qué máquinas se ejecuta el software, no limita el número de instalaciones activas, no permite la desactivación remota de una instalación concreta y no ofrece un registro auditable de los eventos de activación por licencia. La personalización de la página de venta se limita a una plantilla predefinida, aunque esto se mitiga usando la plataforma exclusivamente como pasarela de pago y manteniendo una *landing page* propia.

Shopify [15] es la plataforma de comercio electrónico más consolidada del mercado. No cobra comisión por transacción, pero requiere una suscripción mensual que parte de los 29 \$ mensuales. Esta estructura de costes fijos no se ajusta bien a las primeras etapas de un negocio de creador, en las que los ingresos son variables. Las funcionalidades específicas para productos digitales, como la entrega de archivos o la gestión de licencias, se implementan a través de extensiones de terceros que añaden complejidad y costes adicionales.

La tabla 3.1 recoge una comparativa de las plataformas analizadas en las dimensiones más relevantes para este proyecto.

Plataforma	Comisión	API de verificación	Control por dispositivo	Personalización
Gumroad	10 % + 0,50 \$	Sí (básica)	No	Baja
Lemon Squeezy	5 % + 0,50 \$	Parcial	No	Media
Payhip	5 % (plan libre)	No	No	Baja
Sellfy	0 % + suscripción	No	No	Media
Whop	Variable	Sí (servidor)	No	Baja
Shopify	0 % + suscripción	No	No	Alta

Tabla 3.1: Comparativa de las principales plataformas de venta de productos digitales en las dimensiones relevantes para este proyecto. «Control por dispositivo» indica si la plataforma gestiona el registro, límite y desactivación de máquinas de forma nativa.

Del análisis se desprende que ninguna plataforma cubre de forma completa los requisitos del presente proyecto: soporte para cuatro tipos de producto con comportamiento diferenciado tras la compra, control de activaciones por dispositivo con ciclo de vida completo, panel de administración propio y eliminación de comisiones recurrentes por transacción. Esta brecha justifica el desarrollo de una solución propia en la que el vendedor mantiene control total sobre los datos, la lógica de negocio y la experiencia del usuario.

3.2 Arquitecturas web para aplicaciones de comercio electrónico

3.2.1 Modelos de renderizado

Next.js ofrece tres estrategias de renderizado: Server-Side Rendering (SSR), que genera el HyperText Markup Language (HTML) en el servidor bajo demanda; Client-Side Rendering (CSR), que delega la construcción de la interfaz al navegador; y Static Site Generation (SSG), que pre-genera páginas en tiempo de compilación. El *App Router* de Next.js 15 [16] introduce los React Server Componentss (RSCs): un modelo híbrido en el que los componentes se renderizan en el servidor por defecto y la directiva `"use client"` convierte un componente concreto en interactivo.

Este modelo híbrido resulta idóneo para DigiStore porque unifica en un único repositorio estrategias que el proyecto necesita por separado: el catálogo público de productos requiere SSR para que los motores de búsqueda indexen su contenido y para optimizar el Search Engine Optimization (SEO), mientras que el panel de administración y el proceso de compra requieren CSR para la interactividad. Sin RSCs, satisfacer ambos requisitos habría obligado a mantener proyectos o despliegues separados.

3.2.2 Estilos de API

La API pública de licencias del sistema debe poder ser consumida desde entornos heterogéneos: scripts Python, *expert advisors* escritos en MQL5 ejecutándose dentro de MetaTrader 5 y aplicaciones de escritorio en cualquier plataforma. Esto impone restricciones claras sobre el estilo: mientras que REST funciona desde cualquier entorno capaz de realizar una petición HTTP sin dependencias adicionales, GraphQL [17] requiere un cliente específico en cada entorno consumidor, lo que dificulta su integración en procesos embebidos como los *expert advisors*.

La elección de REST se fundamenta además en la simplicidad del modelo: los tres *endpoints* de verificación, activación y desactivación no presentan el problema de sobretransferencia (*overfetching*) que motivaría GraphQL, y REST con respuestas JavaScript Object Notation (JSON) es el estándar de facto para APIs públicas, lo que facilita su adopción por terceros que consuman el sistema.

3.2.3 Arquitectura monolítica *serverless* vs. microservicios

Una arquitectura de microservicios descompone el sistema en procesos independientes que se escalan y despliegan de forma autónoma [18]. Sus beneficios se materializan en sistemas de gran escala con equipos especializados; en un proyecto de un solo desarrollador y plazo acotado, introduce complejidad operativa (comunicación entre servicios, trazabilidad distribuida y despliegues coordinados) sin aportar valor funcional.

La alternativa adoptada es una arquitectura monolítica integrada en un único proyecto Next.js, desplegada sobre la infraestructura *serverless* de Vercel. Las rutas de API de Next.js actúan como *backend* en el mismo repositorio que el *frontend*, lo que elimina la necesidad de un servidor separado y permite compartir código y tipos entre las capas de presentación y de lógica de negocio [19]. Vercel gestiona el escalado automático de las funciones *serverless*, de modo que el sistema soporta variaciones de carga sin intervención manual sobre la infraestructura.

3.3 Tecnologías del stack seleccionado

3.3.1 Next.js 15

Next.js [16] es el *framework* de desarrollo web basado en React [20] con mayor adopción en el ecosistema JavaScript actual. La versión 15, utilizada en este proyecto, incorpora el *App Router* y los RSCs como unidad de composición fundamental de la interfaz. Esta arquitectura permite colocar la lógica de servidor junto al componente que la consume, eliminando la necesidad de una capa de API explícita para la comunicación interna entre el *frontend* y el *backend*; únicamente los *endpoints* destinados a clientes externos (la API pública de licencias y los *webhooks* de Stripe) se exponen como rutas de API convencionales.

La elección de Next.js responde a tres factores. Primero, su modelo de RSCs y *App Router* permite adoptar SSR para el catálogo público orientado a SEO y CSR para las secciones interactivas, colocando la lógica de servidor junto al componente que la consume dentro de un único repositorio sin capas adicionales. Segundo, la integración nativa con Vercel convierte cada *push* al repositorio remoto en un despliegue automático sin gestión de servidor. Tercero, la experiencia previa del autor con el *framework* reduce el riesgo técnico en un proyecto de duración acotada. Alternativas como Remix, SvelteKit o Nuxt habrían requerido un período de adaptación sin ofrecer ventajas funcionales diferenciadas para este caso de uso concreto.

3.3.2 TypeScript

TypeScript [21] es un superconjunto de JavaScript que añade un sistema de tipos estático: cada variable, parámetro y valor de retorno puede llevar asociado un tipo, y el compilador verifica la coherencia antes de generar el código JavaScript ejecutable. Frente a JavaScript plano, TypeScript desplaza la detección de inconsistencias del tiempo de ejecución al tiempo de compilación, lo que resulta especialmente relevante en un dominio con modelos de datos estrictos: precios en centavos enteros, estados de licencias, identificadores de entidades y formatos de respuesta de APIs externas. Se emplea en la totalidad del código del proyecto, incluida la definición manual de los tipos correspondientes a las tablas de Supabase. Esta práctica convierte los esquemas de la base de datos en contratos verificables estáticamente a lo largo de todo el código, reduciendo la superficie de errores relacionados con la manipulación de datos. Durante el desarrollo se detectó un bug de precios en el que una vista del panel de administración aplicaba dos divisiones consecutivas por cien al formatear los precios, mostrando planes de 30 \$ como 0,30 \$; el sistema de tipos no previno este error concreto, pero la cobertura de TypeScript sobre los tipos de retorno de las funciones de formateo facilitó la identificación y corrección durante la revisión del código.

3.3.3 Supabase

Supabase [22] es una plataforma *backend as a service* (BaaS) construida sobre PostgreSQL que ofrece en una única solución una base de datos relacional, autenticación de usuarios, almacenamiento de archivos y funciones ejecutables en el servidor. A diferencia de gestionar un servidor PostgreSQL propio, Supabase se encarga de la infra-

estructura, los respaldos y las actualizaciones, de modo que el desarrollador trabaja exclusivamente sobre el esquema y los datos.

La elección de Supabase frente a Firebase, la alternativa más conocida en el espacio BaaS, se fundamenta principalmente en el modelo de datos: Firebase emplea una base de datos documental en formato JSON, inadecuada para el esquema relacional de este proyecto, que implica relaciones múltiples entre usuarios, licencias, órdenes, planes, productos y cursos. PostgreSQL ofrece integridad referencial declarativa, transacciones ACID y la capacidad de formular consultas con JOIN, condiciones y agregaciones.

Otros factores determinantes fueron el módulo de autenticación integrado, que gestiona el almacenamiento seguro de contraseñas y el flujo OAuth con Google, y el soporte nativo de Row Level Security (RLS), que permite definir directamente en la base de datos qué filas puede leer o modificar cada usuario. La disponibilidad de servidores en Europa (región eu-west-1, Irlanda) y el generoso nivel gratuito completaron la decisión.

3.3.4 Stripe

Stripe [23] es el procesador de pagos en línea de mayor adopción en el mercado para aplicaciones de internet, con una API que cubre pagos únicos, suscripciones recurrentes, períodos de prueba, reembolsos y gestión de disputas. Se optó por Stripe sin evaluar formalmente alternativas (PayPal, Braintree, Adyen, Paddle) por dos razones: es la referencia del sector en cuanto a calidad de API, documentación y herramientas de desarrollo, y profundizar en su integración formaba parte de los intereses formativos del autor desde el inicio del proyecto.

Un elemento especialmente relevante durante el desarrollo fue la herramienta de línea de comandos oficial de Stripe (`stripe listen`), que permite reenviar al servidor local los eventos de *webhook* del entorno de pruebas mediante un túnel autenticado. Esto eliminó la necesidad de desplegar la aplicación para depurar la lógica de procesamiento de eventos, lo que resultó indispensable dado que prácticamente toda la lógica de licencias (generación tras compra, suspensión por impago, recuperación) se activa a través de *webhooks*.

3.3.5 Tailwind CSS y shadcn/ui

Tailwind CSS [24] es un *framework* de estilos basado en utilidades: en lugar de definir clases semánticas en un fichero CSS separado, los estilos se especifican directamente como clases utilitarias en el marcado del componente. Este enfoque elimina la proliferación de ficheros de estilos, mantiene los estilos colocalizados con el componente que los usa y acelera la iteración sobre el diseño. Se adoptó por experiencia previa del autor y por ser la opción más extendida actualmente en el ecosistema React. El proyecto usa la versión 4 de Tailwind, que adopta una configuración basada en CSS en lugar de un fichero `tailwind.config.js`, y requiere registrar los plugins directamente en el fichero `globals.css` mediante la directiva `@plugin`.

shadcn/ui [25] es una colección de componentes de interfaz contruidos sobre Radix UI y estilizados con Tailwind CSS. A diferencia de una librería de componentes tradicional, shadcn/ui no se instala como dependencia: los componentes se incorporan copiando su código fuente al repositorio, lo que otorga control total sobre su compor-

tamiento y apariencia sin dependencia de actualizaciones externas. Se eligió por su estética adecuada para aplicaciones de tipo *fintech*, su accesibilidad nativa (las primitivas de Radix UI siguen las especificaciones ARIA) y porque todos los componentes están escritos en TypeScript con soporte completo de tipos.

3.3.6 Resend

Resend [26] es un servicio de envío de correo electrónico transaccional con una API diseñada específicamente para el ecosistema React/TypeScript. Su principal diferenciador es la integración con React Email, una librería que permite definir las plantillas de correo electrónico como componentes React. Esto permite usar los mismos patrones de composición en los correos que en la interfaz web, manteniendo la coherencia del código. Se evaluaron como alternativas SendGrid, orientado al envío masivo de marketing con una API más compleja, y nodemailer sobre SMTP, que añade dependencia de infraestructura propia sin garantías de entregabilidad. Resend se eligió por su integración natural con el *stack* del proyecto, su sencillez de configuración y su nivel gratuito suficiente para el volumen de correos transaccionales esperado.

3.4 Seguridad en plataformas de comercio electrónico

El diseño de seguridad de una plataforma de comercio electrónico se enfrenta a un conjunto de amenazas específicas: acceso no autorizado a recursos de otros usuarios, manipulación de precios, interceptación de datos de pago y entrega no controlada de archivos digitales. El *OWASP Top 10* [27], publicado por el *Open Web Application Security Project*, es el marco de referencia más ampliamente reconocido para categorizar y priorizar los riesgos de seguridad en aplicaciones web. En el presente proyecto, este marco se utilizó como guía analítica retrospectiva al redactar la memoria, con el objetivo de verificar que las medidas implementadas cubrían los vectores de ataque más habituales.

A01 - Control de acceso deficiente. La principal amenaza en este apartado es el ataque por referencia directa a objetos inseguros (Insecure Direct Object Reference (IDOR)) [28]: un usuario autenticado accede a recursos de otro usuario manipulando identificadores en las URLs o en el cuerpo de las peticiones. Para mitigarlo, todas las rutas de API que devuelven recursos del usuario filtran explícitamente por el identificador del usuario autenticado además del identificador del recurso solicitado; una petición con credenciales del usuario A sobre un recurso del usuario B devuelve un error 404 en lugar del recurso. Como segunda barrera de defensa, las políticas de RLS de PostgreSQL restringen a nivel de base de datos qué filas puede leer o modificar cada usuario; incluso si una ruta de API omitiera el filtro en el código de aplicación, la base de datos rechazaría el acceso a los datos de otros usuarios.

A02 - Fallos criptográficos. Los archivos digitales vendidos en la plataforma se almacenan en un *bucket* privado de Supabase Storage. Para su descarga, el sistema genera URLs firmadas con un tiempo de expiración de una hora, verificando previamente que el usuario solicitante dispone de una licencia activa para el producto correspondiente.

Esta combinación de autorización explícita y expiración temporal impide tanto el acceso no autorizado como la reutilización indefinida de un enlace de descarga obtenido legítimamente. La terminación TLS se delega en Vercel para el tráfico de la aplicación y en Supabase para las conexiones a la base de datos.

A03 - Inyección. Todas las consultas a la base de datos se realizan a través del cliente JavaScript de Supabase, que utiliza consultas parametrizadas internamente. No se construyen cadenas de consulta SQL mediante concatenación de valores de entrada del usuario, lo que elimina la superficie de ataque de la inyección SQL.

A07 - Fallos de autenticación. La gestión de credenciales y sesiones se delega íntegramente en Supabase Auth, que implementa el almacenamiento seguro de contraseñas mediante *bcrypt* y el flujo *Proof Key for Code Exchange* (Proof Key for Code Exchange (PKCE)) para la autenticación OAuth con Google. El flujo PKCE protege contra ataques de interceptación del código de autorización al añadir un verificador criptográfico al intercambio de tokens, garantizando que solo el cliente que inició el flujo puede completarlo.

A08 - Fallos en la integridad del software y los datos. Los *webhooks* de Stripe se verifican mediante una firma HMAC-SHA256 incluida en la cabecera de cada petición entrante. Antes de procesar cualquier evento de pago (generación de licencias, suspensión por impago, cancelaciones), el manejador verifica que la firma corresponde al secreto de *webhook* configurado. Esto garantiza que los eventos procesados proceden realmente de Stripe y no de peticiones forjadas que intenten modificar el estado de las licencias sin haber realizado un pago real.

Adicionalmente, durante el proceso de desarrollo se identificó un fallo de lógica de negocio en el sistema de períodos de prueba: una funcionalidad de descarte de licencias gratuitas permitía a un usuario reclamar repetidamente el mismo plan de prueba, explotando un control de unicidad insuficiente que solo verificaba licencias en estado activo. La corrección y las decisiones de diseño asociadas se describen con detalle en el capítulo 2.

3.5 Normativa aplicable

Reglamento General de Protección de Datos (RGPD). La plataforma almacena datos de carácter personal de los usuarios: dirección de correo electrónico, nombre completo opcional e identificadores de máquina asociados a las activaciones de licencia. En consecuencia, está sujeta al Reglamento (UE) 2016/679 [29], en vigor desde mayo de 2018 en el ámbito de la Unión Europea.

Para cumplir con el principio de minimización de datos, el sistema no recoge información más allá de lo estrictamente necesario para la prestación del servicio: los datos de tarjeta de crédito se delegan íntegramente en Stripe, que cumple el estándar PCI DSS. La plataforma incluye una política de privacidad accesible públicamente en la ruta `/privacy`, en la que se informa a los usuarios sobre el tratamiento de sus datos. La base de datos de Supabase se configuró en la región `eu-west-1` (Irlanda), lo que

garantiza que los datos de los usuarios europeos se almacenan dentro del territorio de la Unión Europea.

Condiciones de uso de Stripe. Stripe establece en su política de uso aceptable una lista de productos y actividades restringidos, que incluye servicios financieros no regulados y actividades especulativas [30]. El operador de la plataforma debe verificar, antes de la puesta en producción, que los productos comercializados no incurren en dichas restricciones. Como práctica preventiva, la plataforma incorpora advertencias de riesgo en las páginas públicas de los productos de tipo *software* con finalidad de *trading*, informando de que los resultados pasados no garantizan resultados futuros y que la actividad implica riesgo de pérdida del capital invertido.

Facturación electrónica. Stripe genera automáticamente recibos de compra para cada transacción y los envía a la dirección de correo electrónico del comprador, cubriendo el requisito básico de documentación de las transacciones para los usuarios finales. El cálculo y la recaudación automática de impuestos por jurisdicción geográfica quedan fuera del alcance del presente trabajo, como se indicó en la sección de alcance del capítulo introductorio.

ANÁLISIS Y DISEÑO DEL SISTEMA

Este capítulo documenta las decisiones de diseño adoptadas antes de entrar en los detalles de implementación. Se presentan los requisitos del sistema, la arquitectura general, el modelo de datos, el diseño de la API pública y la organización de la interfaz de usuario, con el razonamiento que justifica cada elección relevante.

4.1 Requisitos del sistema

4.1.1 Requisitos funcionales

Los requisitos funcionales se obtuvieron a partir de los objetivos específicos definidos en la introducción y del análisis de las limitaciones de las plataformas existentes, descrito en el capítulo 3. La tabla 4.1 recoge los requisitos agrupados por tipo de actor.

4.1.2 Requisitos no funcionales

La tabla 4.2 recoge los requisitos no funcionales con la métrica o criterio de verificación asociado a cada uno.

ID	Descripción
<i>Usuario no autenticado</i>	
RF-01	Registrarse con correo electrónico y contraseña.
RF-02	Autenticarse con correo y contraseña o mediante OAuth con Google.
RF-03	Recuperar la contraseña a través de un enlace enviado por correo.
RF-04	Consultar el catálogo de productos publicados, filtrable por tipo.
RF-05	Ver la página de detalle de un producto con sus planes y precios.
<i>Usuario autenticado</i>	
RF-06	Comprar un producto seleccionando un plan (único, suscripción o prueba).
RF-07	Aplicar un cupón de descuento durante el proceso de compra.
RF-08	Recibir un correo de confirmación tras completar la compra.
RF-09	Consultar las licencias activas en el panel de usuario.
RF-10	Ocultar o mostrar licencias individuales en el panel de usuario.
RF-11	Cancelar o reactivar una suscripción desde el panel de usuario.
RF-12	Descargar el archivo asociado a un producto comprado (ebook, plantilla).
RF-13	Acceder al visor de cursos con navegación por módulos y lecciones.
RF-14	Marcar lecciones como completadas y consultar el progreso del curso.
RF-15	Recibir notificaciones por correo ante eventos del ciclo de vida de la licencia (pago fallido, cancelación, expiración inminente del periodo de prueba).
<i>Software de terceros (sin sesión de usuario)</i>	
RF-16	Verificar si una clave de licencia es válida y si una máquina está registrada.
RF-17	Registrar una nueva máquina en una licencia (activación).
RF-18	Eliminar el registro de una máquina de una licencia (desactivación).
<i>Administrador</i>	
RF-19	Gestionar el catálogo de productos y sus planes de precio (CRUD completo).
RF-20	Construir el currículo de un curso mediante módulos y lecciones con vídeo, contenido Markdown y archivos adjuntos.
RF-21	Gestionar licencias: consultar, revocar, suspender y reactivar.
RF-22	Consultar todas las órdenes y el detalle de cada una.
RF-23	Gestionar cupones de descuento (crear, activar, desactivar).
RF-24	Moderar reseñas de productos (aprobar o rechazar).

Tabla 4.1: Requisitos funcionales del sistema, agrupados por tipo de actor.

ID	Descripción	Criterio
RNF-01	Latencia de la API pública de licencias	p95 < 200 ms
RNF-02	Cifrado del tráfico en tránsito	HTTPS en todas las rutas
RNF-03	Integridad de los eventos de pago	Verificación Hash-based Message Authentication Code (HMAC)-SHA256 en <i>webhooks</i>
RNF-04	Protección de archivos digitales	URLs firmadas con Time To Live (TTL) máximo de 1 h
RNF-05	Escalabilidad ante picos de tráfico	Escalado automático en Vercel
RNF-06	Protección contra abuso de la API	60 req/min (verify), 30 req/min (activate)
RNF-07	Cumplimiento normativo	Datos almacenados en la UE (eu-west-1)
RNF-08	Compatibilidad de navegadores	Chrome, Firefox y Safari actuales

Tabla 4.2: Requisitos no funcionales del sistema con su criterio de verificación.

4.2 Arquitectura general del sistema

El sistema está formado por cuatro componentes principales. La aplicación Next.js, desplegada en Vercel, constituye la capa de código propio del proyecto: contiene la interfaz web, las rutas de API y los manejadores de eventos de pago. Supabase actúa como plataforma *backend as a service*, aportando la base de datos PostgreSQL, el módulo de autenticación, el almacenamiento de archivos y las funciones programadas en el servidor. Stripe gestiona el procesamiento de pagos. Resend se encarga del envío de correos electrónicos transaccionales. La justificación de la elección de cada uno de estos servicios se presentó en el capítulo 3.

Las interacciones entre componentes siguen dos patrones distintos. El flujo principal es síncrono: el navegador envía una petición HTTP a Next.js en Vercel, un componente de servidor consulta la base de datos de Supabase y devuelve la respuesta al navegador. El flujo de pagos es asíncrono: al completar una compra, Stripe envía un evento al *endpoint* `/api/webhooks/stripe` de la aplicación, que crea la orden, genera la licencia en Supabase y envía el correo de confirmación a través de Resend.

Además de estos dos flujos, una Edge Function alojada en Supabase ejecuta diariamente a las 09:00 UTC para enviar avisos de expiración de periodos de prueba. La función solo invoca el *endpoint* interno `/api/internal/trial-reminder` de la aplicación Next.js, protegido con una cabecera de autenticación. Esta separación entre el disparador programado y la lógica de negocio permite ejecutar el código en el mismo entorno y con las mismas abstracciones que el resto de la aplicación, sin depender de las capacidades de ejecución del entorno Deno de Supabase.

La elección de una arquitectura monolítica en un único repositorio tiene una implicación de diseño relevante: las rutas de API que gestionan el funcionamiento interno no requieren autenticación separada entre capas. Solo los tres *endpoints* de la API pública de licencias y el receptor de *webhooks* de Stripe están diseñados para ser

consumidos desde el exterior.

La tabla 4.3 enumera los cinco eventos de Stripe que procesa el sistema y la acción que dispara cada uno.

Evento de Stripe	Acción en el sistema
<code>checkout.session.completed</code>	Crea la orden, el ítem de orden y la licencia; envía correo de confirmación.
<code>invoice.payment_failed</code>	Pone la licencia en estado <code>suspended</code> ; envía correo de pago fallido.
<code>invoice.payment_succeeded</code>	Reactiva la licencia a estado <code>active</code> ; envía correo de recuperación.
<code>customer.subscription.updated</code>	Sincroniza el campo <code>cancel_at_period_end</code> en la licencia.
<code>customer.subscription.deleted</code>	Marca la licencia como <code>revoked</code> ; envía correo de cancelación.

Tabla 4.3: Eventos de Stripe procesados y acción que dispara cada uno.

4.3 Modelo de datos

El esquema de la base de datos comprende doce tablas agrupadas en tres dominios: el catálogo de productos, el ciclo de vida de las licencias y la gestión de órdenes. La cadena central del modelo es `products` → `license_plans` → `licenses` → `license_activations`: un producto tiene uno o varios planes de precio; al completar una compra se emite una licencia que vincula un usuario con un plan concreto; cada licencia registra las máquinas donde se ha activado como filas individuales en `license_activations`. La analogía más directa es la de un software comercial: el producto es el título, el plan es la modalidad de compra, la licencia es el certificado de propiedad y las activaciones son los equipos donde está instalado.

A continuación se describen las decisiones de diseño más relevantes del esquema.

4.3.1 Planes como entidad propia

Separar `license_plans` de `products` permite que un mismo producto tenga varias modalidades de acceso con comportamiento diferente. Un plan perpetuo no tiene fecha de expiración ni recurrencia en Stripe; un plan de suscripción genera cobros recurrentes y su licencia puede suspenderse por impago; un plan de prueba calcula `expires_at` en el momento de la emisión. Si estas modalidades vivieran directamente en `products`, cada producto necesitaría gestionar todos los tipos de ciclo de vida simultáneamente. Como entidad propia, cada plan tiene su `stripe_price_id` independiente, sus propios valores de `max_activations` y `billing_interval`, y puede activarse o desactivarse sin afectar al resto del catálogo.

4.3.2 Desnormalización del campo `type`

La tabla `licenses` incluye un campo `type` que replica el de `license_plans`. Esta desnormalización responde a dos necesidades. La primera es el rendimiento: filtrar suscripciones activas sin hacer JOIN a `license_plans` reduce la complejidad de las

consultas frecuentes. La segunda es la integridad histórica: los manejadores de eventos de Stripe leen `licenses.type` directamente para decidir qué acción ejecutar, sin necesidad de acceder al plan original. Si el tipo de un plan cambiara en el futuro, las licencias ya emitidas seguirían reflejando el tipo con el que fueron compradas.

4.3.3 Contador de activaciones y tabla de activaciones

La tabla `licenses` mantiene un campo `activation_count` junto a la tabla `license_activations`. El contador sirve para la verificación rápida: comprobar si `activation_count < max_activations` es una lectura directa sobre un campo indexado, sin necesidad de agregar filas. La tabla de activaciones, en cambio, almacena el detalle de cada máquina registrada (`machine_id`, nombre, dirección IP, `last_seen_at`), que usa el panel de administración para mostrar qué equipos tiene un usuario y el *endpoint* de desactivación para eliminar un registro concreto. Los dos conviven porque sirven propósitos distintos: velocidad el contador, detalle la tabla.

4.3.4 Registro de eventos como log de auditoría

La tabla `license_events` registra cada cambio de estado de una licencia (emisión, activación, desactivación, suspensión, expiración, revocación, renovación), junto con la dirección IP y el `machine_id` involucrado cuando aplica. A diferencia del estado actual almacenado en `licenses`, este registro permite reconstruir el historial completo de una licencia. Durante el desarrollo resultó útil para depurar el flujo de *webhooks* y rastrear el ciclo de vida de las licencias de prueba.

4.3.5 Precios en centavos enteros

Los campos de precio (`license_plans.price`, `orders.total_amount`) se almacenan como enteros en la unidad mínima de la moneda. Los números en coma flotante IEEE 754 no representan exactamente valores decimales como 0,1, y eso acumula errores en operaciones aritméticas monetarias. La API de Stripe usa la misma convención: el campo `unit_amount` espera centavos [23]. Durante el desarrollo se detectó un bug en el panel de administración donde dos divisiones consecutivas por cien mostraban planes de 30 \$ como 0,30 \$; la conversión a la unidad de presentación ocurre exclusivamente en la capa de vista.

4.3.6 Separación entre órdenes e ítems de orden

La separación entre `orders` y `order_items` cumple dos funciones. Primero, preserva el precio pagado en el momento de la compra: si el vendedor modifica el precio de un plan, el historial de órdenes refleja lo que el usuario pagó realmente. Segundo, la estructura permite añadir en el futuro un carrito de múltiples productos por pedido sin modificar el esquema. En la versión actual, cada orden contiene exactamente un ítem.

4.3.7 Tabla profiles

Supabase gestiona los usuarios en el esquema `auth`, que no es accesible para consultas del esquema `public` a través de PostgREST [22]. La tabla `profiles` replica los campos

necesarios (`email`, `full_name`) y añade los propios de la aplicación: `role` (`admin` o `customer`) y `avatar_url`. Un trigger de PostgreSQL (`on_auth_user_created`) copia los datos del nuevo usuario desde `auth.users` en el momento del registro, garantizando que `profiles` esté siempre sincronizada sin lógica adicional en el código de aplicación.

4.3.8 Identificadores públicos con *slug*

Los productos se identifican públicamente mediante un campo `slug` legible (por ejemplo, `/products/bot-pro`), mientras que todas las tablas usan UUID como clave primaria. Los *slugs* incluyen palabras clave en la URL y mejoran el posicionamiento en motores de búsqueda; además, los usuarios que comparten un enlace perciben el nombre del producto directamente en la dirección. Los UUID garantizan unicidad global sin colisiones y evitan exponer identificadores secuenciales que revelarían el volumen de la base de datos.

4.4 API pública de licencias

La API pública expone tres *endpoints* destinados a ser consumidos desde aplicaciones de terceros, sin sesión de usuario. La tabla 4.4 los enumera con los campos del cuerpo de la petición.

Endpoint	Descripción	Cuerpo (JSON)
POST <code>/api/v1/licenses/verify</code>	Comprueba si la licencia es válida	<code>license_key</code> , <code>machine_id</code> ?
POST <code>/api/v1/licenses/activate</code>	Registra una máquina en la licencia	<code>license_key</code> , <code>machine_id</code> , <code>machine_name</code> ?
POST <code>/api/v1/licenses/deactivate</code>	Elimina el registro de una máquina	<code>license_key</code> , <code>machine_id</code>

Tabla 4.4: Endpoints de la API pública de licencias v1. Los tres usan el método POST y devuelven JSON.

4.4.1 Semántica de verificar y activar

Los tres *endpoints* tienen semánticas distintas. `verify` es una operación de solo lectura: comprueba el estado de la licencia y, si se proporciona `machine_id`, verifica que esa máquina ya está registrada; actualiza el campo `last_seen_at` pero no crea nada nuevo. `activate` es una operación de escritura: registra la máquina en `license_activations`, incrementa `activation_count` y falla si se alcanzó el límite de activaciones. El flujo habitual de una aplicación cliente es llamar a `activate` la primera vez que se ejecuta en un equipo nuevo, y llamar a `verify` en cada arranque posterior.

4.4.2 La clave de licencia como credencial

Los *endpoints* no requieren sesión de usuario ni clave de API adicional. La `license_key` actúa directamente como credencial: es una cadena de veinte caracteres hexadecimales

agrupados en cuatro bloques (AAAAA-BBBBB-CCCCC-DDDDD), generada con el módulo `crypto` de Node.js [31], que usa un generador de números pseudoaleatorios criptográficamente seguro (CSPRNG). Quien posee la clave está autorizado a usarla. Este diseño reduce la fricción de integración, en particular para clientes en lenguajes con soporte limitado para la gestión de múltiples credenciales, como los *expert advisors* escritos en MQL5.

El formato de cuatro bloques separados por guiones también tiene ventajas prácticas: es más legible que un UUID continuo, facilita la escritura manual cuando sea necesaria y es reconocible visualmente como clave de licencia.

4.4.3 Identificador de máquina generado en el cliente

El campo `machine_id` lo genera el software cliente, no el servidor. La convención habitual es calcular un *hash* de características hardware estables de la máquina (número de serie del procesador, dirección MAC de la interfaz de red principal o una combinación de ambos). El servidor acepta cualquier cadena y confía en que el cliente la genera de forma consistente. Esta responsabilidad recae en el desarrollador que integra la API, lo que da flexibilidad para adaptar la generación al entorno de cada aplicación.

4.4.4 Limitación de tasa y política de caché

Los *endpoints* aplican limitación de tasa por dirección IP con umbrales distintos. `verify` permite hasta 60 peticiones por minuto: se llama potencialmente en cada arranque del software y es de solo lectura. `activate` permite hasta 30 peticiones por minuto: se llama raramente (solo al instalar en un equipo nuevo) pero tiene efectos de escritura.

La política de caché de las respuestas es asimétrica. Las respuestas que indican licencia válida llevan `Cache-Control: no-store`: una licencia puede ser revocada en cualquier momento, por lo que cachear una respuesta positiva comprometería la seguridad. Las respuestas negativas (licencia no encontrada, revocada o expirada) llevan `Cache-Control: max-age=30`: son estables a corto plazo, y cachearlas en la red de distribución de Vercel reduce la carga sobre la base de datos ante intentos repetidos con claves inválidas.

4.5 Diseño de la interfaz de usuario

La interfaz organiza la experiencia en tres zonas con niveles de acceso distintos.

4.5.1 Tienda pública

La tienda pública (`/products` y `/products/[slug]`) es accesible sin autenticación. Las páginas se renderizan en el servidor para que los motores de búsqueda indexen el contenido del catálogo [16]. Cada página de producto muestra los planes disponibles con sus precios, características y botón de compra. Si el usuario no tiene sesión activa, el flujo de compra redirige al formulario de acceso y devuelve al usuario a la misma página tras autenticarse.

4.5.2 Flujo de compra

El flujo de compra sigue estos pasos desde la perspectiva del usuario: seleccionar el producto y el plan, aplicar opcionalmente un cupón, y confirmar la compra. Si el plan tiene precio distinto de cero, el servidor crea una sesión de *checkout* en Stripe y redirige al navegador a la página de pago alojada por Stripe; si el plan es gratuito o de prueba, la licencia se genera directamente sin pasar por Stripe. Tras el pago, Stripe redirige al usuario a `/checkout/success` y envía simultáneamente el evento `checkout.session.completed` al sistema, que crea la licencia en base de datos y envía el correo de confirmación. El usuario puede acceder a su nueva licencia desde el panel de usuario en segundos.

4.5.3 Panel de usuario

El panel de usuario (`/dashboard`) requiere autenticación y agrupa cuatro secciones: licencias, órdenes, cursos y perfil. La sección de licencias fue la más compleja de diseñar en términos de experiencia de usuario. Cada tarjeta de licencia muestra información adaptada al tipo de producto: el número de activaciones y la clave para licencias de software; un botón de descarga para ebooks y plantillas; un acceso al visor de cursos con el porcentaje de progreso para cursos. Los usuarios pueden marcar licencias como ocultas para retirarlas de la vista principal sin eliminarlas, con la opción de recuperarlas a través del parámetro `?show_hidden=1` en la URL. Esto evita saturar el panel con licencias de prueba expiradas o productos que el usuario ya no usa.

4.5.4 Visor de cursos

El visor de cursos (`/dashboard/courses/[productId]`) presenta una columna lateral con el currículum del curso (módulos y lecciones) y un área principal con el reproductor de vídeo, el contenido en Markdown y los archivos adjuntos de la lección activa. Esta estructura, habitual en plataformas de cursos de referencia como Udemy o Teachable, tiene una ventaja técnica adicional: permite asociar el marcado de lección completada a una unidad discreta, lo que simplifica el seguimiento del progreso.

Los vídeos se sirven mediante *embed* de YouTube o Vimeo en lugar de alojarlos en Supabase Storage. Almacenar vídeo propio implicaría costes de almacenamiento y ancho de banda significativos, así como la necesidad de gestionar la transcodificación a múltiples resoluciones. Las plataformas de vídeo existentes ya resuelven ambos problemas sin coste adicional para el administrador.

4.5.5 Panel de administración

El panel de administración (`/admin`) requiere el rol `admin` en la tabla `profiles` y cubre siete secciones: productos, planes, licencias, órdenes, cupones, reseñas y clientes. El diseño siguió un enfoque iterativo sin *wireframes* previos, tomando como referencia la organización de plataformas de administración consolidadas (Whop, Shopify) y los componentes de tabla y formulario de *shadcn/ui* [25].

La lista de licencias del panel experimentó una revisión relevante durante el desarrollo. La primera versión solo mostraba la clave de licencia y el número de activaciones. Al identificarse la necesidad de localizar licencias por cliente, se añadió la columna de

email del usuario y una búsqueda dual: si el término de búsqueda contiene arroba, el sistema busca en `profiles` por email y filtra las licencias por los identificadores resultantes; de lo contrario, busca por clave de licencia. Esta lógica requirió dos consultas separadas, ya que PostgREST no puede realizar *joins* entre tablas del esquema `public` y `auth.users` [22].

4.6 Control de acceso y seguridad en el diseño

Las medidas de seguridad concretas se describieron en el capítulo 3. Desde la perspectiva del diseño, dos decisiones estructurales condicionan cómo se implementan esas medidas.

La primera es la separación de los clientes de Supabase según el contexto de ejecución. El cliente estándar, sujeto a las políticas de RLS, se usa en componentes de usuario y en las rutas de API que procesan peticiones autenticadas. El cliente de rol de servicio, que omite RLS, se usa exclusivamente en los manejadores de *webhooks* y en operaciones de administración, donde no hay sesión de usuario activa. Esta separación garantiza que las políticas de RLS actúen como segunda barrera para el código de usuario sin interferir con las operaciones del sistema.

La segunda es el campo de rol almacenado en `profiles`. La política de actualización de RLS sobre `profiles` permite a cada usuario modificar su propia fila, incluyendo técnicamente el campo `role`. La única protección actual es que el código de la aplicación nunca incluye `role` en el *payload* de actualización del formulario de perfil. Una solución más robusta requeriría restricciones a nivel de columna en PostgreSQL o un trigger `BEFORE UPDATE` que ignore cambios en `role` desde sesiones que no sean de rol de servicio. Esta limitación queda identificada como trabajo futuro en el capítulo 8.

IMPLEMENTACIÓN

Este capítulo describe las decisiones de implementación más relevantes del sistema, con especial atención a los problemas técnicos encontrados durante el desarrollo y a las soluciones adoptadas. No pretende ser un recorrido exhaustivo por el código, sino documentar los aspectos donde las decisiones tuvieron mayor impacto sobre la arquitectura, la seguridad o el comportamiento del sistema en producción.

5.1 Organización del código y convenciones

5.1.1 Clientes de Supabase según el contexto de ejecución

La aplicación usa tres clientes de Supabase distintos, cada uno con credenciales y propósito diferente. `createClient()` es asíncrono y lee las cookies de la petición HTTP para obtener la sesión del usuario activo; utiliza la clave anónima, de modo que todas las consultas pasan por las políticas de RLS y solo ven los datos del usuario autenticado. `createServiceClient()` usa la clave de rol de servicio y *bypasa* RLS completamente; se emplea en páginas de administración y en rutas de API que necesitan acceder a datos de todos los usuarios. `createAdminClient()` tiene la misma autoridad que el anterior pero se inicializa con `autoRefreshToken: false` y `persistSession: false`, porque en los manejadores de *webhooks* no existe una sesión de usuario que mantener activa [22].

Esta separación no es solo organizativa: mezclar credenciales distintas en un único cliente crearía un riesgo de escalada de privilegios. El cliente de usuario sujeto a RLS actúa como segunda barrera para el código de la aplicación; el de servicio se reserva para los contextos donde esa barrera es innecesaria o contraproducente.

5.1.2 Tipos de base de datos escritos manualmente

Los tipos de las tablas de Supabase se definieron manualmente en `types/database.ts` en lugar de generarlos con la CLI de Supabase. La razón fue técnica: la versión `@supabase/supabase-js v2.98.0`

requiere un campo `Relationships: []` en cada definición de tabla; los tipos generados por la CLI en ese momento no lo incluían, lo que provocaba que los tipos `Insert` y `Update` se resolvieran a `never` e inutilizaran el autocompletado. La definición manual garantizó el formato exacto que la librería esperaba. Sin estos tipos, TypeScript no puede verificar nombres de columnas incorrectos ni tipos incompatibles en tiempo de compilación; esos errores solo emergen en ejecución.

5.1.3 Validación con Zod

La validación de formularios y de cuerpos de peticiones usa esquemas Zod [14]. La alternativa habitual es validar manualmente con `if/throw`, pero esa aproximación no aporta información de tipo al compilador: el dato sigue siendo `unknown` después de las comprobaciones. Con Zod, al llamar `schema.parse(body)` el resultado tiene el tipo exacto inferido del esquema, con verificación estática en todo el código siguiente. Esto permite compartir el mismo esquema entre el formulario del cliente (integrado con `react-hook-form` para mensajes de error automáticos) y la ruta de API que recibe los mismos datos, sin duplicar las reglas de validación. Los esquemas se centralizan en `lib/utils/validators.ts` y cubren tanto los formularios de administración (`productSchema`, `licensePlanSchema`) como los cuerpos de la API pública (`verifyLicenseSchema`, `activateLicenseSchema`, `deactivateLicenseSchema`).

Un ajuste necesario durante el desarrollo: los esquemas con `.default()` de Zod convertían el tipo de entrada a `T | undefined`, lo que conflictuaba con `react-hook-form`. Se eliminaron los defaults de Zod y se movieron al parámetro `defaultValues` de `useForm`.

5.2 Autenticación y gestión de sesiones

5.2.1 Flujo OAuth con PKCE y callback

La autenticación con Google sigue el flujo PKCE, cuya justificación de seguridad se describió en el capítulo 3. Desde el punto de vista de la implementación, el aspecto clave es que Google no devuelve los tokens directamente al navegador: redirige al callback `/auth/callback` con un código de un solo uso. Sin este endpoint, el código en la URL de redirección no tendría ningún handler que lo intercambiara por tokens reales, y el usuario llegaría al *dashboard* sin sesión activa. El handler llama a `supabase.auth.exchangeCodeForSession(code)`, que intercambia el código por los tokens y los almacena en cookies. El fragmento 5.1 muestra su parte central.

```
1 const supabase = createServerClient(url, anonKey, { cookies:
  { ... } })
2 const { error } = await supabase.auth.exchangeCodeForSession(
  code)
3 if (!error) return NextResponse.redirect(new URL(destination,
  request.url))
4 return NextResponse.redirect(new URL('/auth/login?error=
  auth_error', origin))
```

Listing 5.1: Intercambio del código OAuth por sesión en `/auth/callback`.

PKCE protege el intercambio porque el cliente genera un `code_verifier` secreto al iniciar el flujo y envía a Google solo su hash (`code_challenge`). Al completar el intercambio en el callback, debe demostrar que conoce el verificador original, de modo que interceptar el código en la URL de redirección no permite completar la autenticación.

5.2.2 Trigger de creación de perfil

Al registrar un usuario, el `full_name` se incluye en el campo `raw_user_meta_data` de Supabase Auth, y un trigger de PostgreSQL (`on_auth_user_created`) lo copia automáticamente a `profiles`. Esto garantiza que el perfil se crea en la misma operación que el usuario, sin ventana de inconsistencia. Si los dos inserts se hicieran desde el código de la aplicación, cualquier fallo entre ambos dejaría un usuario sin perfil y la aplicación fallaría en cualquier página que lo consulte. El trigger también cubre la creación de usuarios vía Google OAuth, donde no pasa por el formulario de registro.

5.2.3 Bug crítico: recursión infinita en las políticas RLS

Durante el desarrollo, el panel de administración mostraba datos vacíos aunque existían registros en la base de datos. El error en los logs era `42P17: infinite recursion detected in policy for relation "profiles"`.

La causa fue la política RLS «Admin reads all profiles», que verificaba el rol con:

```
1 EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role
        = 'admin')
```

Listing 5.2: Política RLS con recursión (versión incorrecta).

Al evaluar esta política para cualquier consulta sobre `profiles`, PostgreSQL necesitaba consultar `profiles`, lo que disparaba la misma política otra vez, indefinidamente. El problema se amplificaba porque todas las políticas de admin en otras tablas usaban la misma subconsulta: cualquier *query* de un usuario admin desencadenaba el ciclo. La solución fue eliminar esa política y migrar todas las páginas y rutas de administración a `createServiceClient()`, que *bypasa* RLS completamente. La enseñanza es que las políticas RLS no pueden hacer subconsultas recursivas sobre la misma tabla ni sobre tablas cuyas políticas dependan de la tabla original; la forma correcta de verificar el rol de admin en RLS es usar `auth.jwt()` para leer *claims* del token.

5.3 Integración con Stripe

5.3.1 Bifurcación en el flujo de compra

La ruta `/api/checkout` tiene dos caminos según el tipo de plan. Para planes gratuitos o trials sin tarjeta, la licencia se genera directamente y se devuelve en la misma respuesta HTTP; el usuario la ve disponible de forma inmediata, sin pasar por la página alojada de Stripe. Para planes de pago o suscripciones con trial nativo, se crea una Checkout Session de Stripe y se devuelve la URL de pago. Esto evita crear sesiones de pago de \$0 (confusas para el usuario) y elimina la latencia del ciclo *webhook* para productos que no requieren cobro real.

5.3.2 Verificación HMAC de los webhooks

Los *webhooks* de Stripe se verifican mediante firma HMAC-SHA256 antes de procesar cualquier evento. El listado 5.3 muestra el mecanismo:

```
1 const body = await request.text() // body crudo, antes de
   parsear
2 const sig = request.headers.get('stripe-signature')
3 if (!sig) return new Response('Missing signature', { status:
   400 })
4
5 let event: Stripe.Event
6 try {
7   event = stripe.webhooks.constructEvent(
8     body, sig, process.env.STRIPE_WEBHOOK_SECRET!
9   )
10 } catch {
11   return new Response('Invalid signature', { status: 400 })
12 }
```

Listing 5.3: Verificación de firma HMAC en el receptor de webhooks.

Sin esta verificación, cualquiera que conozca la URL del *endpoint* podría enviar un evento `checkout.session.completed` falso con un `user_id` y un `product_id` arbitrarios, generando una licencia activa sin ningún pago real. El body se lee como texto antes de procesarlo porque parsear a JSON primero puede alterar la serialización y hacer que la firma calculada no coincida. Este mecanismo implementa el control A08 del Open Web Application Security Project (OWASP) Top 10 [27].

Los manejadores de eventos usan `createAdminClient()` porque los *webhooks* son peticiones de los servidores de Stripe, sin sesión de usuario activa. El cliente estándar intentaría leer cookies para construir la sesión y, al no encontrarlas, devolvería consultas vacías por efecto de RLS.

5.3.3 Diagnóstico de un cambio silencioso en la API de Stripe v2026

La versión 2026-02-25.clover de la API de Stripe eliminó el campo `invoice.subscription` y movió el identificador de suscripción a `invoice.parent.subscription_details.subscription`. El problema fue que el fallo era completamente silencioso: los manejadores de pago fallido y de pago recuperado obtenían `subscriptionId = undefined`, buscaban una licencia con ese ID, no encontraban ninguna, y retornaban sin modificar nada. Ningún error era visible en los logs. El síntoma se detectó al probar manualmente el flujo con `stripe listen` y observar que el estado de la licencia no cambiaba tras simular un pago fallido [23]. La corrección fue actualizar la ruta de acceso al campo en ambos manejadores.

5.4 Ciclo de vida de licencias

5.4.1 Estados de una licencia

Toda licencia del sistema tiene en todo momento uno de cinco estados posibles. La tabla 5.1 los enumera con su descripción y la transición que los origina.

Estado	Descripción	Origen
active	Licencia válida y en uso.	Compra completada o pago de renovación.
trial	Período de prueba activo; acceso completo hasta <code>expires_at</code> .	Emisión de plan trial o inicio de suscripción con trial nativo.
suspended	Suspensión temporal por fallo de pago; Stripe reintentará el cobro.	Evento <code>invoice.payment_failed</code> .
expired	El período de la licencia o del trial ha vencido.	Paso de <code>expires_at</code> o evento de fin de suscripción.
revoked	Revocación administrativa permanente.	Acción manual del admin o evento <code>customer.subscription.deleted</code> .

Tabla 5.1: Estados posibles de una licencia y la transición que origina cada uno.

5.4.2 Generación de claves

El diseño del formato de clave se justificó en el capítulo 4. La función `generateLicenseKey()` del listado 5.4 implementa ese diseño:

```

1 import { randomBytes } from 'crypto'
2
3 export function generateLicenseKey(): string {
4   const hex = randomBytes(10).toString('hex').toUpperCase()
5   return `${hex.slice(0,5)}-${hex.slice(5,10)}-${hex.slice(10,15)}-${hex.slice(15,20)}`
6 }

```

Listing 5.4: Generador de claves de licencia con entropía criptográfica.

El módulo `crypto` de Node.js [31] usa un generador de números pseudoaleatorios criptográficamente seguro (CSPRNG), lo que garantiza que la clave no es predecible aunque se conozca la secuencia de claves anteriores. Los 10 bytes aleatorios equivalen a 80 bits de entropía, suficiente para que un ataque de fuerza bruta sea computacionalmente inviable. La función `generateUniqueLicenseKey` envuelve al generador y verifica colisiones en base de datos con hasta diez reintentos, aunque la probabilidad de colisión con este espacio de claves es prácticamente nula.

5.4.3 Corrección del exploit de periodos de prueba

El capítulo 2 mencionó que el sistema de trials pasó por varias iteraciones. La vulnerabilidad concreta fue la siguiente.

En una versión intermedia del desarrollo se implementó un botón «Discard license» que ponía la licencia en estado `revoked`. El control de unicidad del trial buscaba licencias con `status IN ('active', 'trial')`: una licencia revocada no bloqueaba la re-obtención del mismo plan de prueba, de modo que un usuario podía descartar el trial y reclamarlo de nuevo indefinidamente.

La corrección cambió la consulta para buscar cualquier licencia histórica con `type = 'trial'` para ese plan y usuario, independientemente del estado:

```

1 const { data: anyTrialLicense } = await supabaseAdmin
2   .from('licenses')

```

```
3   .select('id')
4   .eq('user_id', user.id)
5   .eq('license_plan_id', planId)
6   .eq('type', 'trial')
7   .maybeSingle()
8
9   if (anyTrialLicense) {
10    return NextResponse.json(
11      { error: 'You have already used your trial for this plan'
12        },
13      { status: 409 }
14    )
15  }
```

Listing 5.5: Control de unicidad de trial corregido: consulta histórica sin filtro de estado.

En sesiones posteriores se eliminó el botón «Discard license» completamente. La decisión respondió a dos razones. La primera fue técnica: la función era la raíz de toda la complejidad acumulada en el sistema de trials y seguía dejando abierta la posibilidad de futuros exploits similares. La segunda fue conceptual: ninguna plataforma de referencia del sector (Gumroad [2], Lemon Squeezy [3], Payhip [4]) implementa la noción de descartar una licencia. Una licencia gratuita simplemente existe en la biblioteca del usuario; si no la usa, la ignora. Añadir ese comportamiento creaba una expectativa que no existe en el sector y añadía fricción innecesaria.

5.4.4 Estados de licencia ante fallos de pago

Cuando Stripe no puede cobrar una renovación de suscripción, el evento `invoice.payment_failed` pone la licencia en estado `suspended` en lugar de `expired` o `revoked`. La diferencia semántica es relevante: `suspended` es temporal y reversible. Si el usuario actualiza su método de pago y Stripe cobra exitosamente en un reintento posterior, el evento `invoice.payment_succeeded` reactiva la licencia a `active` sin intervención manual. Usar `expired` o `revoked` habría cortado el acceso de forma definitiva, penalizando al usuario durante el período de reintentos en el que Stripe todavía puede recuperar el cobro.

5.4.5 Sincronización de `cancel_at_period_end`

El campo `cancel_at_period_end` en `licenses` se sincroniza exclusivamente desde el evento `customer.subscription.updated` de Stripe, no desde la respuesta de la llamada a la API de cancelación. Esto garantiza que la base de datos refleja exactamente lo que Stripe registra, independientemente de desde dónde se inició la cancelación: si el usuario cancela directamente desde el panel de Stripe, el `webhook` llega igualmente y actualiza el estado local.

5.5 Entrega de contenido según el tipo de producto

El comportamiento posterior a la compra varía según el tipo de producto. Para los productos de tipo *software*, el acceso consiste en una clave de licencia activable mediante la API pública; el usuario la ve en el *dashboard* y la introduce en la aplicación

descargada por otros medios. Para los productos de tipo *ebook* y *template*, el sistema genera bajo demanda una URL firmada que apunta al archivo en Supabase Storage; la descarga caduca al cabo de una hora. Para los productos de tipo *course*, el acceso da entrada al visor integrado donde el usuario consume el contenido directamente en el navegador, con seguimiento de progreso por lección.

5.5.1 Descarga de archivos con URL firmada

La justificación de seguridad de las URLs firmadas se presentó en el capítulo 3. La implementación en el *endpoint* GET `/api/downloads/[productId]` añade una capa de autorización específica del dominio: antes de generar la URL, verifica que el usuario tiene sesión activa, que el producto existe y tiene un `file_path` asociado, y que existe una licencia en estado `active` o `trial` para ese producto filtrada por `user_id` y `product_id`. Este último filtro previene IDOR [28]: una petición con credenciales del usuario A sobre un producto del usuario B devuelve 403 en lugar del archivo. Solo si las tres verificaciones pasan se genera la URL firmada con TTL de una hora y `download: true`.

5.5.2 Validación de progreso en cursos

El *endpoint* POST `/api/courses/[productId]/progress` aplica la misma defensa en profundidad. Antes de registrar una lección como completada, verifica que el usuario tiene licencia activa para el producto y que la lección pertenece al producto indicado en la URL. Este segundo control evita que un usuario con licencia para un curso marque lecciones de otro curso usando un `lesson_id` externo. La operación usa `upsert` con `ignoreDuplicates: true`, lo que hace la llamada idempotente: marcar dos veces la misma lección como completada no genera un error ni una fila duplicada.

5.6 Comunicaciones y tareas programadas

5.6.1 Cinco eventos de correo transaccional

El sistema envía correos ante cinco eventos del ciclo de vida de una licencia. La confirmación de compra se despacha desde `handleCheckoutSessionCompleted`. El correo de pago fallido se despacha desde `handleInvoicePaymentFailed`. La recuperación de pago se despacha desde `handleInvoicePaymentSucceeded`, pero solo si el estado previo de la licencia era `suspended`, para no enviar el correo en cada renovación normal. El aviso de cancelación se despacha desde `handleSubscriptionUpdated`, solo cuando `cancel_at_period_end` cambia de `false` a `true`. El recordatorio de `trial` por expirar lo gestiona el *endpoint* interno, y solo se envía una vez por licencia, marcando `trial_reminder_sent: true` en el campo `metadata` para evitar reenvíos.

Las funciones de envío protegen su ejecución con `try/catch` y silencian los errores mediante `console.error`. Esto garantiza que cualquier fallo de red o de Resend no propague una excepción al manejador de *webhook*, lo que haría que Stripe reintentara el evento indefinidamente al recibir un código de error.

5.6.2 Arquitectura de dos pasos para el cron de trials

El cron diario de recordatorios de trial está implementado como una Edge Function de Supabase que llama a un *endpoint* interno de Next.js, en lugar de ejecutar la lógica completa en la propia función. El motivo fue técnico: `@react-email/render` depende de `react-dom/server`, que no está disponible en el runtime Deno de las Edge Functions de Supabase. Reimplementar el renderizado de plantillas en Deno habría duplicado toda la lógica de emails. Con el patrón de dos pasos, la Edge Function actúa como dispatcher; su único trabajo es hacer una llamada HTTP al *endpoint* interno con el header de autenticación `x-internal-secret`. Toda la lógica de consulta, renderizado y envío vive en Node.js, donde React Email funciona sin restricciones.

Se evaluaron como alternativas Vercel Cron Jobs (descartada por crear dependencia de la plataforma de despliegue) y `pg_cron` de PostgreSQL (descartada porque no puede hacer llamadas HTTP a servicios externos directamente).

5.7 Incompatibilidad entre Edge Runtime y el cliente de Supabase

Durante el desarrollo del *middleware* de protección de rutas se detectó un bug crítico: al acceder a `/admin` con una cuenta de administrador autenticada mediante Google OAuth, el *middleware* redirigía al *dashboard* aunque la sesión existía correctamente. Los registros mostraban que `getUser()` devolvía al usuario, pero la consulta posterior a *profiles* para verificar el rol retornaba `null` sin lanzar ningún error.

La causa fue una incompatibilidad entre el cliente `@supabase/ssr` y el Edge Runtime de Vercel cuando la sesión se había creado mediante Google OAuth. El cliente intentaba acceder a módulos de Node.js no disponibles en ese entorno. La solución fue reemplazar la llamada al cliente Supabase por un `fetch` nativo a la REST API de Supabase, usando la clave de servicio directamente:

```
1  const res = await fetch(  
2    `${supabaseUrl}/rest/v1/profiles?id=eq.${user.id}&select=  
      role`,  
3    {  
4      headers: {  
5        apikey: serviceKey,  
6        Authorization: `Bearer ${serviceKey}`,  
7      },  
8    }  
9  )  
10 const rows: { role: string }[] = await res.json()  
11 if (rows[0]?.role !== 'admin') { /* redirect */ }
```

Listing 5.6: Verificación de rol en Edge Runtime mediante `fetch` nativo.

El `fetch` nativo es Hypertext Transfer Protocol (HTTP) puro y funciona en cualquier runtime. Como consecuencia de este problema, el manejador de *webhooks* de Stripe tiene la directiva `export const runtime = 'nodejs'` explícita, porque la verificación HMAC usa el módulo `crypto` de Node.js, que tampoco está disponible en el Edge Runtime [31].

5.8 Patrón de dos consultas para licencias de administrador

Cuando el administrador necesita resolver una incidencia de soporte o revocar el acceso de un cliente, la búsqueda natural es por dirección de correo electrónico. Sin embargo, PostgREST no puede realizar *joins* entre tablas del esquema `public` y la tabla `auth.users` del esquema `auth` [22], por lo que no es posible filtrar licencias por email en una sola consulta. La solución usa dos consultas separadas:

```

1 // 1. Obtener licencias
2 const { data: licenses } = await admin.from('licenses').
  select('*')...
3
4 // 2. Obtener perfiles de los usuarios involucrados
5 const userIds = [...new Set(licenses.map(l => l.user_id))]
6 const { data: profiles } = await admin
7   .from('profiles').select('id, email, full_name').in('id',
  userIds)
8
9 // 3. Mapa para lookup en O(1)
10 const profileMap = Object.fromEntries(profiles.map(p => [p.id
  , p]))

```

Listing 5.7: Patrón de dos consultas para combinar licencias con perfiles de usuario.

Para la búsqueda por correo electrónico el proceso se invierte: si el término contiene @, se busca primero en `profiles` por email, se extraen los `user_id` que coinciden, y se filtra la consulta de licencias con `.in('user_id', ids)`. Si ningún perfil coincide, se devuelve la lista vacía sin hacer la segunda consulta.

5.9 Limitaciones técnicas identificadas

Dos decisiones de implementación funcionan correctamente en el entorno de desarrollo y en producción con tráfico bajo, pero presentan limitaciones conocidas que conviene documentar.

La primera es el *rate limiting* de la API pública de licencias, implementado con un Map en memoria de JavaScript. En un despliegue serverless de Vercel con escalado horizontal, cada instancia de función tiene su propio mapa completamente aislado. Con tráfico alto, un atacante podría enviar peticiones a múltiples instancias en paralelo, cada una con su propio contador, y no ser bloqueado efectivamente por ninguna. La solución correcta requiere un almacén compartido y persistente como Upstash Redis.

La segunda es el campo `activation_count` en `licenses`, que se incrementa y decrementa manualmente en el código de la aplicación. Si una operación fallara a mitad (actualizando el contador pero no la tabla `license_activations`, o viceversa), el contador quedaría desincronizado con las filas reales. La solución más robusta sería derivar el contador de un `COUNT(*)` sobre `license_activations` en tiempo de consulta, o usar una columna generada por PostgreSQL. La implementación actual se mantiene porque las operaciones son simples y la probabilidad de fallo parcial es baja, pero constituye una deuda técnica identificada.

PRUEBAS Y VALIDACIÓN

Este capítulo describe el proceso de verificación de la plataforma: la estrategia adoptada, los escenarios funcionales y de seguridad que se ejecutaron, las métricas de rendimiento obtenidas y los defectos detectados durante el desarrollo.

6.1 Estrategia de pruebas

El proyecto no incorpora una suite de tests automatizados. La decisión respondió a la naturaleza de la aplicación: la mayor parte de la lógica de negocio reside en la interacción con servicios externos (Supabase con RLS activo, Stripe en modo test, Supabase Storage). Cubrir esa lógica con tests unitarios requeriría mocks que replicarían el comportamiento de esos servicios, reduciendo la fidelidad de las pruebas respecto al sistema real. El cambio silencioso en la API de Stripe v2026, analizado en el apartado 5.3.3, ilustra este punto: una versión nueva de la API cambió la ubicación de un campo en el objeto de factura y los manejadores de *webhook* dejaron de encontrar el identificador de suscripción, sin que se produjera ningún error. Un mock de la API de Stripe nunca habría reproducido ese cambio, porque lo que falló era el comportamiento del servicio real, no el del código propio.

La verificación se organizó en tres capas. La primera, estática, operó durante la escritura del código: TypeScript verificó los tipos en compilación y ESLint detectó patrones problemáticos antes de ejecutar nada. La segunda consistió en ejecutar la aplicación en modo desarrollo local (`npm run dev`) y ejercitar cada flujo en el navegador, con los logs del servidor de Next.js y la consola del navegador monitorizados en paralelo. Para los flujos con Stripe, el CLI de Stripe reenviaba los webhooks al servidor local mediante `stripe listen --forward-to localhost:3000/api/webhooks/stripe`, permitiendo probar el ciclo completo de pago sin exponer el servidor a internet. La tercera capa se aplicó al endpoint de mayor demanda potencial: el script `scripts/load-test.mjs`, que mide la latencia y la estabilidad del endpoint de verificación de licencias bajo peticiones concurrentes.

6.2 Verificación estática

TypeScript actuó como primera barrera de detección. Dos casos concretos ilustran su utilidad. El primero fue la incompatibilidad entre `.default()` de Zod y react-hook-form: añadir ese método a un campo del esquema convertía el tipo de entrada en `T | undefined`, lo que generaba errores de tipo en el `register` de react-hook-form visibles directamente en el editor antes de ejecutar ningún código. El segundo ocurrió cuando los tipos de `types/database.ts` carecían del campo `Relationships: []`: el compilador resolvía los tipos `Insert` y `Update` a `never`, rechazando cualquier operación de escritura sobre la base de datos. Ambos errores habrían pasado desapercibidos hasta tiempo de ejecución en un lenguaje sin tipado estático.

El proceso de compilación completo (`next build`) sirvió como capa adicional. En la cuarta sesión de desarrollo, el componente `DeleteProductButton.tsx` importaba `@/components/ui/alert-dialog`, que no estaba instalado. El servidor de desarrollo en modo `next dev` no alertó del error; fue el build quien lo detectó al resolver todos los imports del grafo de dependencias. La solución fue instalar el componente con `npx shadcn@latest add alert-dialog`.

El valor de esta capa es que desplaza la detección de errores al momento más temprano y barato posible: un error de tipos o de import se corrige en segundos en el editor, mientras que el mismo fallo descubierto en ejecución exige reproducir el estado que lo provoca. No sustituye a las pruebas funcionales, porque no comprueba que la lógica de negocio sea correcta, solo que sea consistente; pero elimina de raíz una familia entera de errores antes de que el código llegue siquiera a ejecutarse.

6.3 Pruebas funcionales

Los flujos principales de la plataforma se verificaron de forma manual en el entorno de desarrollo local, usando cuentas de prueba creadas ad hoc y la tarjeta de prueba de Stripe 4242 4242 4242 4242 para los pagos. La tabla 6.1 recoge los doce escenarios ejecutados y sus resultados.

Los doce casos se completaron con el resultado esperado. Conviene precisar el alcance de esta cobertura: los escenarios recorren el camino principal de cada flujo (el llamado *happy path*) y los puntos de bifurcación más importantes, como la distinción entre compra de pago y activación gratuita, o el bloqueo del segundo intento de trial. No agotan, en cambio, el espacio de casos de error de cada flujo, que se aborda de forma separada en los apartados 6.4 y 6.7. La elección de qué escenarios incluir priorizó aquellos en los que un fallo tendría consecuencias directas para el usuario o para el negocio: la generación incorrecta de una licencia, un cobro equivocado o la pérdida del progreso en un curso.

Tabla 6.1: Casos de prueba funcionales ejecutados.

ID	Escenario	Condición	Resultado
CP-01	Registro con email	Email nuevo, contraseña válida	Cuenta creada, email de verificación recibido
CP-02	Registro con email duplicado	Email ya registrado en el sistema	Mensaje de error en el campo del formulario
CP-03	Autenticación con Google OAuth	Cuenta Google personal del desarrollador	Sesión iniciada, fila creada en perfiles
CP-04	Compra de pago único	Tarjeta de prueba Stripe, plan perpetuo	Licencia active, email de confirmación recibido
CP-05	Compra de producto gratuito	Plan con price = 0	Licencia active sin intervención de Stripe
CP-06	Activación de trial gratuito	Plan de tipo trial	Licencia trial generada, plan marcado como usado
CP-07	Reintento de trial ya consumido	Segunda activación del mismo trial	Error 409 con mensaje de error correcto en la interfaz
CP-08	Suscripción con trial nativo	Tarjeta requerida, cobro diferido	Licencia trial, suscripción en estado <i>trialing</i> en Stripe
CP-09	Cancelación de suscripción	Botón de cancelación en el panel	cancel_at_period_end = true, etiqueta <i>Cancels on...</i>
CP-10	Descarga de archivo	Licencia activa, producto con archivo	URL firmada generada, descarga iniciada
CP-11	Seguimiento de progreso en curso	Marcar lección como completada y recargar	Estado persiste entre sesiones
CP-12	Búsqueda de licencias en admin	Búsqueda por email y por clave de licencia	Resultados correctos en ambos modos

6.4 Pruebas de seguridad

La verificación de los controles de acceso combinó pruebas manuales con revisión del código. La tabla 6.2 recoge los seis escenarios considerados.

Tabla 6.2: Escenarios de seguridad verificados.

ID	Escenario	Método	Resultado
CS-01	Descarga sin licencia (IDOR)	Petición directa a <code>/api/downloads/[productId]</code> con cuenta sin licencia	Error 403 con mensaje JSON
CS-02	Acceso a licencia ajena (IDOR)	Navegación a <code>/dashboard/licenses/[uuid-ajeno]</code> con sesión distinta	Error 404
CS-03	Acceso al admin sin rol	Navegación directa a <code>/admin/products</code> con cuenta de usuario	Redirección automática a <code>/dashboard</code>
CS-04	Explotación del trial	Ciclo: activar trial, descartar, reactivar	Error 409; exploit documentado y corregido en §5.4.3
CS-05	Webhook sin firma HMAC	Revisión del handler de webhooks	<code>constructEvent</code> lanza excepción si la firma no es válida
CS-06	Acceso directo a archivos	Revisión de la configuración de Storage	Bucket privado; petición sin URL firmada devuelve 400

Los casos CS-01 y CS-02 se probaron manualmente con dos cuentas de usuario distintas, copiando el identificador de un recurso de la primera e intentando acceder a él desde la segunda. CS-01 confirmó que la ruta `/api/downloads/[productId]` verifica la licencia activa antes de generar la URL firmada: la cuenta sin licencia recibió una respuesta 403 con el mensaje *No active license found for this product*, sin que se llegara a generar ninguna URL de descarga. CS-02 devolvió un 404 porque la *query* que carga la licencia filtra por `id` y `user_id` simultáneamente, de modo que el Universally Unique Identifier (UUID) de una licencia ajena no produce ningún resultado. El sistema no distingue entre «el recurso no existe» y «el recurso no es tuyo», lo que evita además revelar la existencia de recursos de otros usuarios. CS-03 se verificó de forma implícita durante la depuración de la Sesión 4, cuando se corrigió la lógica de verificación de roles en el middleware. CS-05 y CS-06 se verificaron por revisión directa del código: el handler de webhooks llama a `stripe.webhooks.constructEvent(body, sig, secret)`, que lanza una excepción si la firma no coincide con el secreto configurado [23], y el bucket de Storage se definió como privado en la migración inicial, por lo que cualquier acceso directo sin URL firmada es rechazado por la propia capa de almacenamiento.

6.5 Pruebas de rendimiento

6.5.1 Prueba de carga sobre la API pública

El endpoint de verificación de licencias (`POST /api/v1/licenses/verify`) es el punto de mayor demanda potencial del sistema: cada aplicación de software vendida lo consulta cada vez que un usuario la ejecuta. Para medir su comportamiento bajo carga concurrente, se desarrolló el script `scripts/load-test.mjs`, que lanza N peticiones en paralelo durante T segundos y calcula los percentiles de latencia. La prueba se ejecutó con concurrencia 20 y duración 30 segundos sobre el servidor de desarrollo local (WSL2, modo `npm run dev`). Los resultados figuran en la tabla 6.3.

Tabla 6.3: Resultados de la prueba de carga sobre `POST /api/v1/licenses/verify` (concurrencia 20, 30 s).

Métrica	Valor (ms)
Mínimo	55,7
Media	101,5
p50	90,2
p95	127,3
p99	394,5
Máximo	2 722,4

El test emitió 5 912 peticiones con una tasa de error del 0 %, a un ritmo de 197 req/s. El objetivo de $p95 < 100$ ms no se alcanzó: el valor registrado fue 127,3 ms. La desviación se atribuye a las condiciones del entorno de prueba: el modo de desarrollo de Next.js incluye compilación TypeScript por petición y omite las optimizaciones de producción; WSL2 añade latencia de red y de sistema de archivos respecto a un servidor nativo Linux. Los picos en p99 y máximo (394,5 ms y 2 722,4 ms) son consistentes con pausas de recolección de basura del *runtime* de Node.js en modo desarrollo. En un entorno de producción con la compilación optimizada (`next build`), el *connection pooling* de Supabase y la ejecución en Vercel, las latencias serían distintas.

6.5.2 Análisis con Lighthouse

Las dos páginas de mayor tráfico público se analizaron con la herramienta Lighthouse de Chrome en modo escritorio. La tabla 6.4 recoge las puntuaciones.

Tabla 6.4: Puntuaciones Lighthouse en las páginas principales (escritorio).

Página	Rendimiento	Accesibilidad	Buenas prácticas	SEO
Inicio (/)	98	94	100	100
Catálogo (/products)	99	89	100	100

Las puntuaciones de rendimiento (98 y 99 sobre 100) reflejan las optimizaciones que Next.js aplica en modo producción: renderizado en servidor, carga diferida de imágenes

y eliminación de Cascading Style Sheets (CSS) no utilizado por parte de Tailwind. El tiempo hasta el primer elemento visible (FCP) fue de 0,3 s en ambas páginas; el tiempo hasta el elemento de mayor tamaño (LCP), de 1,1 s en la página de inicio y 0,9 s en el catálogo. El bloqueo del hilo principal (TBT) fue 0 ms y la inestabilidad de maquetación (CLS), 0 en los dos casos. Las puntuaciones de accesibilidad (94 y 89) indican que la mayoría de criterios de accesibilidad se cumplen, con margen de mejora en el contraste de algunos elementos secundarios de la interfaz.

6.6 Defectos detectados durante el desarrollo

La verificación manual descrita en este capítulo no solo confirmó comportamientos correctos: también sacó a la luz un conjunto de defectos que se corrigieron durante el desarrollo. La tabla 6.5 los consolida como registro de incidencias, indicando para cada uno su síntoma observable, la causa que se identificó al investigarlo y la solución aplicada. Dos de ellos comparten un rasgo revelador sobre la dificultad de probar este tipo de sistema: tanto D-02 como D-04 son fallos silenciosos que no producían ninguna excepción ni mensaje en los logs. En D-02 el panel de administración mostraba un precio incorrecto por una doble conversión de unidades; en D-04 un manejador de *webhook* dejaba de actualizar el estado de una licencia al no encontrar un campo que la API de Stripe había movido de sitio. En ninguno de los dos casos había nada visiblemente roto: solo se detectaban al comparar el resultado observado con el esperado. Esa clase de fallo es la que mejor justifica la verificación manual de extremo a extremo, frente a una batería de pruebas que solo comprueban que el código no lanza errores. El detalle técnico de los defectos D-01, D-03 y D-04, por su impacto sobre la arquitectura, se desarrolla en el capítulo 5.

Tabla 6.5: Defectos detectados durante el proceso de desarrollo.

ID	Síntoma	Causa raíz	Solución
D-01	Panel de admin mostraba datos vacíos	Política RLS con subconsulta recursiva (error 42P17)	Migrar consultas de admin a <code>createServiceClient()</code>
D-02	El panel de administración mostraba un precio 100 veces menor	Doble división por 100 al formatear un valor ya almacenado en centavos	Pasar el precio a <code>formatCurrency</code> sin dividir
D-03	Trial reutilizable de forma indefinida	El check solo miraba licencias activas, no el historial completo	Buscar cualquier licencia histórica con <code>type='trial'</code>
D-04	El estado de una licencia no se actualizaba tras un pago en Stripe v2026	La API movió <code>invoice.subscription</code> a una ruta anidada; el handler recibía un identificador indefinido	Actualizar la ruta de acceso al campo en los manejadores
D-05	OAuth ignoraba el parámetro <code>redirectTo</code>	Prioridad incorrecta de redirección en el callback handler	Leer <code>searchParam redirectTo</code> antes del fallback
D-06	Error de compilación en producción	Componente <code>alert-dialog</code> importado pero no instalado	Instalar con <code>npx shadcn@latest add alert-dialog</code>
D-07	LicenseCard mostraba «Expires» en suscripciones activas	Lógica de etiqueta no contemplaba <code>cancel_at_period_end</code>	Añadir condición: suscripción activa sin cancelación pendiente muestra «Renews»

6.7 Gestión de errores y casos límite

Los flujos que se ejercitan en condiciones normales no cubren todos los comportamientos relevantes del sistema. Cinco casos adicionales corresponden a situaciones excepcionales cuyo manejo correcto es parte del diseño: un webhook que Stripe reintenta, un usuario que intenta abusar del período de prueba, una licencia que expira sin que ningún proceso la marque, un cliente que envía demasiadas peticiones, y un entorno de desarrollo sin credenciales de correo configuradas. La tabla 6.6 recoge estos comportamientos y el método con que se verificaron.

Tabla 6.6: Comportamiento del sistema ante condiciones excepcionales.

Condición	Comportamiento esperado	Verificación
Webhook de Stripe reintentado	No se crea una segunda licencia; el handler comprueba si ya existe antes de insertar	Revisión del código de <code>handleCheckoutSession-Completed</code>
Trial ya consumido (segundo intento de compra)	Respuesta 409 con el mensaje exacto del cuerpo JSON mostrado en la interfaz	Probado manualmente (CP-07)
Licencia expirada sin actualizar en BD	El endpoint <code>/api/v1/licenses/verify</code> detecta <code>expires_at</code> pasado, actualiza el estado a <code>expired</code> y devuelve <code>valid: false</code> en la misma petición	Revisión del handler de verificación
Rate limit excedido en la API pública	Respuesta 429; el cliente debe implementar reintentos con retroceso	Revisión del middleware de rate limiting
RESEND_API_KEY no configurado	Las funciones de envío retornan sin lanzar ningún error; se emite un aviso en consola	Revisión de <code>lib/email/send.tsx</code> ; permite desarrollar localmente sin cuenta Resend activa

6.8 Limitaciones del proceso de pruebas

La estrategia de verificación manual por capas adoptada en este proyecto tiene cuatro limitaciones que conviene reconocer de forma explícita. Para cada una se indica no solo en qué consiste, sino por qué resultó aceptable dentro del alcance del trabajo.

La primera es que la prueba de carga se ejecutó sobre el servidor de desarrollo local y no sobre el entorno de producción en Vercel. El objetivo de esa prueba era detectar cuellos de botella en la lógica del endpoint y comprobar que se mantuviera estable bajo concurrencia, no certificar un tiempo de respuesta de producción: medir esto último exigiría una instancia desplegada con tráfico representativo y el *connection pooling* de Supabase configurado, lo que constituye un trabajo de validación de infraestructura distinto del que aquí se aborda. Por eso el umbral de $p95 < 100$ ms se interpreta como una referencia de diseño, no como un criterio incumplido.

La segunda es que la prevención de referencias directas a objetos se verificó con pruebas manuales y revisión del código, sin herramientas de análisis de seguridad especializadas como OWASP ZAP o Burp Suite. Una auditoría formal es una disciplina

en sí misma; en su lugar, el proyecto mitigó por diseño el riesgo principal (el control de acceso roto, categoría A01 de OWASP, tratada en el capítulo 3) haciendo que toda ruta filtre por `user_id`, y ejercitó manualmente los vectores más explotables. Esto ofrece una confianza razonable para el modelo de amenazas de la aplicación, aunque no equivalga a una certificación de seguridad.

La tercera es que la plataforma se desarrolló y verificó en Chrome, asumiendo la compatibilidad con Firefox y Safari sin probarla de forma sistemática. La razón por la que esta omisión tiene bajo riesgo es que el stack (Tailwind CSS y los componentes de Radix UI) genera CSS y marcado estándar con amplia cobertura de navegadores documentada; una matriz de pruebas cruzada probablemente no habría alterado ninguna decisión de diseño.

La cuarta es que dos flujos de Stripe, el ciclo de suscripción con prueba hasta el cobro automático al vencer y el flujo de pago fallido seguido de recuperación, se verificaron por revisión de los manejadores de webhook y no por ejecución en tiempo real. Ambos dependen de eventos disparados por el tiempo que controla Stripe (la expiración del trial, el reintento de cobro), cuya reproducción exigiría manipular el reloj de la pasarela o esperar días reales; además, los manejadores implicados comparten la misma ruta de código que sí se ejercitó con los eventos que se simulaban manualmente, lo que reduce el riesgo de que un fallo quede sin detectar.

RESULTADOS Y DISCUSIÓN

7.1 Funcionalidades implementadas

[Insertar tabla de funcionalidades implementadas aquí.]

7.2 Métricas de rendimiento

[Insertar métricas de rendimiento aquí.]

7.3 Capturas del sistema en funcionamiento

[Insertar capturas de pantalla aquí.]

7.4 Comparación con soluciones existentes

[Insertar tabla comparativa y análisis aquí.]

7.5 Discusión

[Redactar discusión aquí.]

CONCLUSIONES Y TRABAJO FUTURO

8.1 Cumplimiento de objetivos

[Redactar evaluación del cumplimiento de objetivos aquí.]

8.2 Conclusiones principales

[Redactar conclusiones principales aquí.]

8.3 Limitaciones del trabajo

[Redactar descripción de las limitaciones aquí.]

8.4 Trabajo futuro

[Redactar propuestas de trabajo futuro aquí.]



MANUAL DE INSTALACIÓN Y DESPLIEGUE

A.1 Prerrequisitos

[Redactar lista de prerrequisitos aquí.]

A.2 Configuración de variables de entorno

[Redactar descripción de variables de entorno aquí.]

A.3 Instalación y ejecución local

[Redactar instrucciones de instalación local aquí.]

A.4 Migraciones de base de datos

[Redactar instrucciones de migración aquí.]

A.5 Despliegue en Vercel

[Redactar instrucciones de despliegue aquí.]



DOCUMENTACIÓN DE LA API PÚBLICA

B.1 Verificar licencia

[Redactar documentación completa del endpoint verify aquí.]

B.2 Activar licencia

[Redactar documentación del endpoint activate aquí.]

B.3 Desactivar licencia

[Redactar documentación del endpoint deactivate aquí.]

B.4 Ejemplo de integración en Python

[Insertar ejemplo de integración aquí.]

ESQUEMA COMPLETO DE LA BASE DE DATOS

[Insertar esquema completo o diagrama ER de alta resolución aquí.]

BIBLIOGRAFÍA

- [1] Whop Corp., “Digital product statistics: Key trends and data,” <https://whop.com/blog/digital-product-statistics/>, 2025, consultado en 2025. 1
- [2] Gumroad Inc., “Gumroad — sell what you know,” <https://gumroad.com>, 2024, consultado en 2025. 1, 3.1, 5.4.3
- [3] Lemon Squeezy LLC, “Lemon squeezy — payments, subscriptions and digital products,” <https://www.lemonsqueezy.com>, 2024, consultado en 2025. 1, 3.1, 5.4.3
- [4] Payhip Ltd., “Payhip — sell digital downloads & online courses,” <https://payhip.com>, 2024, consultado en 2025. 1, 3.1, 5.4.3
- [5] Sellfy Inc., “Sellfy — build your online store,” <https://sellfy.com>, 2024, consultado en 2025. 1
- [6] Whop Corp., “Whop — the internet’s marketplace for memberships and software,” <https://whop.com>, 2024, consultado en 2025. 1, 3.1
- [7] Gumroad Inc., “Gumroad pricing,” <https://gumroad.com/pricing>, 2025, consultado en 2025. 1, 3.1
- [8] Lemon Squeezy LLC, “Lemon squeezy pricing,” <https://www.lemonsqueezy.com/pricing>, 2025, consultado en 2025. 1, 3.1
- [9] N. Nolan, “Gumroad prices are going up - here are the top 5 alternatives,” <https://medium.com/@nick-nolan/gumroad-prices-are-going-up-here-are-the-top-5-alternatives-231782e1161>, 2023, consultado en 2025. 1
- [10] Trustpilot, “Lemon squeezy - customer reviews,” <https://www.trustpilot.com/review/lemonsqueezy.com>, 2024, consultado en 2025. 1
- [11] Microsoft Corporation, “Visual studio code,” <https://code.visualstudio.com>, 2024, consultado en 2025. 2.3
- [12] Software Freedom Conservancy, “Git,” <https://git-scm.com>, 2024, consultado en 2025. 2.3
- [13] OpenJS Foundation, “ESLint - pluggable javascript linter,” <https://eslint.org>, 2024, consultado en 2025. 2.3

- [14] Colin McDonnell, “Zod - typescript-first schema validation with static type inference,” <https://zod.dev>, 2024, consultado en 2025. 1, 5.1.3
- [15] Shopify Inc., “Shopify — build your business,” <https://www.shopify.com>, 2024, consultado en 2025. 3.1
- [16] Vercel Inc., “Next.js documentation,” <https://nextjs.org/docs>, 2024, consultado en 2025. 3.2.1, 3.3.1, 4.5.1
- [17] GraphQL Foundation, “GraphQL: A query language for your API,” <https://graphql.org>, 2024, consultado en 2025. 3.2.2
- [18] S. Newman, *Building Microservices*. O’Reilly Media, 2015. 3.2.3
- [19] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. 3.2.3
- [20] Meta Platforms Inc., “React documentation,” <https://react.dev>, 2024, consultado en 2025. 3.3.1
- [21] Microsoft Corporation, “TypeScript documentation,” <https://www.typescriptlang.org/docs>, 2024, consultado en 2025. 3.3.2
- [22] Supabase Inc., “Supabase documentation,” <https://supabase.com/docs>, 2024, consultado en 2025. 3.3.3, 4.3.7, 4.5.5, 5.1.1, 5.8
- [23] Stripe Inc., “Stripe api reference,” <https://stripe.com/docs/api>, 2024, consultado en 2025. 3.3.4, 4.3.5, 5.3.3, 6.4
- [24] Tailwind Labs Inc., “Tailwind CSS documentation,” <https://tailwindcss.com/docs>, 2024, consultado en 2025. 3.3.5
- [25] Shadcn, “shadcn/ui — beautifully designed components built with radix ui and tailwind CSS,” <https://ui.shadcn.com>, 2024, consultado en 2025. 3.3.5, 4.5.5
- [26] Resend Inc., “Resend documentation,” <https://resend.com/docs>, 2024, consultado en 2025. 3.3.6
- [27] OWASP Foundation, “OWASP top ten 2021,” <https://owasp.org/Top10>, 2021, consultado en 2025. 3.4, 5.3.2
- [28] —, “Insecure direct object reference prevention cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html, 2024, consultado en 2025. 3.4, 5.5.1
- [29] Parlamento Europeo y Consejo de la Unión Europea, “Reglamento (ue) 2016/679 del parlamento europeo y del consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales,” Diario Oficial de la Unión Europea, L 119, 2016. 3.5
- [30] Stripe Inc., “Stripe services agreement and restricted businesses policy,” <https://stripe.com/legal/restricted-businesses>, 2024, consultado en 2025. 3.5

- [31] OpenJS Foundation, “Node.js crypto module documentation,” <https://nodejs.org/api/crypto.html>, 2024, consultado en 2025. 4.4.2, 5.4.2, 5.7